

# KRCIGE：一个有界工程框架

以认识、资源、控制与恢复约束解释软件工程决策

作者信息待补

单位与电子邮箱待补

理论化修订 v0.3

2026 年 9 月 12 日

## 摘要

软件工程积累了大量有条件成立的经验原则，但在具体决策中，团队仍需说明其收益来源、实施成本和失效边界。本文提出 KRCIGE，作为组织这些判断的条件性解释与决策框架。框架包含认识有限（K）、资源有限（R）、控制有限（C）与工程不可逆性（I）四个描述性要素，工程目标（G）与伦理边界（E）两个规范性要素，以及支持有效反馈的环境结构条件  $\Sigma$ 。

本文以情境化序贯决策模型连接信息价值、预算约束、策略风险及任务相关的恢复成本，重点明确恢复对象、允许误差、成功概率和证据范围。十三条中间原则给出从机制到工程实践的条件性解释；少数形式命题在明确前提下证明，其余机制保留为待检验假设。框架的理论位置建立在有限理性、软件工程经济学、实物期权、面向恢复的计算和社会技术系统研究之上。

合成发布案例及参数敏感性计算展示了观察、扩大与停止之间的策略反转。《程序员修炼之道》100 条 Tips 与 Twelve-Factor App 的映射仅作为解释性材料；本文尚未完成独立的前瞻性预测或因果效果评估。后续检验的重点是：显式列出恢复负担、利益相关者和约束条件，能否改善真实团队的方案识别、边界判断与决策质量。

**关键词：**软件工程理论；有限理性；序贯决策；工程不可逆性；恢复成本；可检验性

## 1 引言

软件工程长期关注可靠性、经济性、演化与协作。早期 NATO 会议将经济地构建可靠软件置于工程议程之中 [17]；IEEE 术语标准强调系统化、规范化和可度量的开发与维护过程 [18]；Boehm 将成本估计和经济权衡用于软件工程决策 [19]；《Software Engineering at Google》进一步强调软件跨时间、规模与组织的演化 [20]。这些传统已经提供了丰富理论与方法。

本文关注的问题是：面对 DRY、低耦合、小步迭代、版本控制、自动化与持续反馈等实践，如何在同一工程情境中说明它们为何有用、需要付出什么成本，以及何时应当停止或改用其他方案？相关的经济分析并非空白。软件实物期权研究已把信息隐藏、分阶段开发与承诺时机联系到不确定条件下的价值 [21]；CBAM 已将架构候选方案、成本收益和有限资源纳入结构化决策 [22]。KRCIGE 的出发点是对这些思路进行有边界的整合与操作化。

框架使用以下组织结构：

情境、目标与约束  $\rightsquigarrow$  条件性作用机制  $\rightsquigarrow$  候选策略比较。

其中 K、R、C、I 分别提示决策者检查信息缺口、资源预算、控制范围及恢复负担；G 与 E 明确评价目标及允许边界； $\Sigma$  要求反馈具有可识别的适用结构。本文的贡献定位为：

- (1) **统一描述**：用任务、主体、视界及策略模型连接上述问题，并区分某项策略的实际风险与可行策略风险的下界；
- (2) **恢复操作化**：以任务等价恢复的最低预期成本  $I_{\varepsilon, \delta}$  明确恢复对象、证据、容差与可靠性要求，讨论其与技术债务的联系和区别；
- (3) **条件与检验**：把十三条工程原则组织为机制假设，利用可复算案例展示边界，并提出有基线、有反例的独立评估协议。

上述贡献属于解释与决策辅助层面。标准的信息价值、动态规划和风险优化结果构成所用工具，本文不主张这些数学机制为新发现，也不主张四个要素已经构成最小、独立或完备的公理集。按照 Gregor 对理论类型的区分，解释、预测和设计行动具有不同的证据要求 [29]；本稿主要提供条件性解释与规范性决策模型，实际预测能力和团队使用效果仍需独立研究。

## 2 KRCIGE 理论框架与基本模型

**定义 2.1** (工程情境). 工程情境记为

$$\mathcal{S}_Q = (Q, s, \tau, O, \mathcal{A}_s, T, B, \mathcal{A}_E).$$

其中，任务  $Q$  明确了系统的状态空间  $\mathcal{W}$ 、预期交付价值度量、损失函数及状态恢复要求；主体  $s$  规定了控制权限与观测接口；有限时间窗  $\tau = \{0, \dots, H\}$  构成了评估系统交付、反馈与恢复的时域视界。 $B$  为初始可用资源向量， $\mathcal{A}_s$  为主体权限允许的全部行动空间， $\mathcal{A}_E(h_t) \subseteq \mathcal{A}_s$  表示在历史轨迹  $h_t$  下满足伦理与合规边界的可行行动子集。

状态演化、观测与行动选择满足

$$w_{t+1} = T(w_t, a_t, d_t), \quad o_t = O(w_t), \quad h_t = (o_0, a_0, \dots, a_{t-1}, o_t), \quad a_t = \pi(h_t). \quad (2.1)$$

非完全可观测状态通过观测核  $P(o_t | w_t)$  产生观测信号； $d_t$  代表不可控的环境扰动， $\mu_t = P(w_t | h_t)$  为主体基于历史信息构建的后验信念状态。策略  $\pi$  依据截止当前的历史信息序贯选择下一步行动。定义  $c_t \geq 0$  为单步资源消耗向量， $b_t = B - \sum_{\ell < t} c_\ell$  为当前时刻的剩余预算，可行策略空间定义为

$$\Pi_Q(h_t, b_t) = \left\{ \pi : a_\ell = \pi(h_\ell) \in \mathcal{A}_E(h_\ell), \sum_{\ell=t}^{H-1} c_\ell \preceq b_t \text{ 几乎处处} \right\}. \quad (2.2)$$

初始决策根节点简记为  $\Pi_Q(B) = \Pi_Q(h_0, B)$ ，其中  $h_0 = (o_0)$ 。资源预算具有多维分量量纲，涵盖工期、算力、存储配额、财务预算、团队注意力以及协作工时等。视具体工程场景，亦可引入机会约束（概率预算）或期望预算作为扩展约束形式。

**定义 2.2** (轨迹价值与可行决策). 令  $\omega = (w_0, a_0, \dots, w_H)$  为系统演化轨迹， $v_t$  为当期交付价值， $\ell_t$  为直接后果损失， $\kappa_t$  为行动消耗折算的资源成本， $v_H$  为评估终期的终止残值。定义

$$U_Q(\omega) = \sum_{t=0}^{H-1} \zeta^t (v_t - \ell_t - \kappa_t) + \zeta^H v_H, \quad 0 < \zeta \leq 1, \quad (2.3)$$

$$V_Q(h_t, b_t) = \sup_{\pi \in \Pi_Q(h_t, b_t)} \mathbb{E}[U_{Q,t:H}(\pi) | h_t]. \quad (2.4)$$

$\Pi_Q(h_t, b_t)$  表示从历史  $h_t$  继发执行的可行策略集合。最优价值由上确界给出：当最优策略存在时，记为  $\pi_Q^*$ 。若任务另行规定最坏期望风险上限，应采用第 5.3 节定义的受约束策略集  $\Pi_Q^g$  与值函数  $V_Q^g$ 。在状态、行动及观测空间有限，转移与观测模型已给定，且采用充分的信念与预算状态时，基本模型可借助动态规划后向递推精确求解；增加策略级风险约束后，还需保留评估该约束所需的状态或直接优化整段策略。

其中， $\ell_t$  度量系统缺陷对用户、业务及可用性造成的直接损失；修复与恢复操作所占用的工时、算力及协调资源计入  $\kappa_t$ 。各项代价在工程价值账本中清晰归属。资源预算不仅界定了工程方案的物理可行域，其对偶影子价格亦反映了资源占用的机会成本。伦理与合规边界以刚性可行集约束体现；对于多目标决策情境，则借助预设的效用偏好序或 Pareto 占优前沿进行仲裁。

K、R、C、I 构成了剖析同一工程情境的四维诊断参量： $k_Q$  具象化为决策相关的预期信息价值； $B_j$  与  $\lambda_j$  分别代表第  $j$  类资源的绝对存量及其边际影子价格； $\gamma_Q$  衡量可行策略的最坏期望风险下确界，具体策略的风险记为  $r_Q(\pi)$ ； $I_{\epsilon, \delta}$  则直接对应恢复至任务等价状态的最低预期成本。在进行跨项目对比时，需基准化评估时间窗与价值折算尺度，或公开标定转换参数与测量误差。恢复成本经折算后进入轨迹效用  $U_Q$ ，而恢复状态的精度与成功概率则作为可行性硬约束发挥作用。

**注 2.3** (时间折现、视界截断与现实工程张力). 式(2.3) 的长程轨迹价值采用离散时间折现累加形式，这提供了一个表达 Google 关于“软件工程是在时间维度上积分的编程” (*Programming integrated over time* [20]) 这一工程主张的模型形式。现实中的有限时域折现求和并非平滑无摩擦的数学积分；在将该形式化模型映射至工业实践时，必须面对以下三项源于有界理性的结构性现实张力：

- (1) **视界截断**：有限时间窗  $\tau = \{0, \dots, H\}$  把超出  $H$  的未来损益概括为终止残值  $\zeta^H v_H$ 。若评估只覆盖单次交付，又在残值中遗漏遗留的恢复义务，模型可能低估当期防护投入的价值，把恢复成本转嫁至后续周期。这是一种可能的成本转移机制；其是否解释实际债务累积，还取决于目标、激励和残值估计。
- (2) **环境与依赖漂移**：模型中的转移规则与扰动集合只在声明的适用条件内成立。海勒姆定律强调可观测行为形成外部依赖的工程风险 [20]。依赖增加可能使同一变更更难撤回，但兼容迁移或权限变化也可能降低恢复成本；I 和  $\gamma_Q$  的时间方向需要重新评估。
- (3) **主体交接**：人员更替或团队重组可能使后继主体无法继承完整的历史证据，从而改变其诊断与决策能力。知识外部化 (P12)、显式假设 (P5) 和证据保留 (P6) 是候选应对措施，其价值取决于信息是否可用以及维护成本，不能只由任期长度推出。

### 3 推导语义

**形式蕴含。**  $A \implies B$  表示在给定定义和数学前提下的严格逻辑后果。命题随文登记可行域、概率模型与量纲，并提供证明。

**条件性工程压力与比较静态。**  $A \rightsquigarrow P_i$  表示情境条件  $A$  通过明确机制改变实践  $P_i$  相对于基线策略  $\pi_0$  的相对优势。令  $\theta \in \{k_Q, \lambda_j, \gamma_Q, I_{\epsilon, \delta}\}$  为情境状态参数，定义实践  $\pi_i$  相对于替代策略  $\pi_0$  的期望净价值差为

$$\Delta V_i(\theta) = J_Q(\pi_i; \theta) - J_Q(\pi_0; \theta).$$

这里  $J_Q(\pi; \theta) = \mathbb{E}[U_Q(\pi; \theta)]$  表示给定策略的价值，与对全部可行策略取上确界的  $V_Q$  区分。比较静态命题需指定参数化的情境族、固定的其他条件，以及两项策略在比较区间内的可行性； $k_Q$ 、

$\gamma_Q$  等派生量并不总能独立改变。若已证明  $\Delta V_i$  关于指定参数弱单调递增, 才可报告相应的单调结论; 仅有定性机制时,  $\rightsquigarrow$  表示待检验的方向假设。

例如, 固定故障概率  $p$ , 若防护能降低固定比例  $\xi$  的恢复成本  $I$ , 额外成本为  $C$ , 且不改变其他后果, 则  $\Delta V = p\xi I - C$ , 因而  $\partial \Delta V / \partial I = p\xi$ 。若防护效果、故障概率或可行策略随  $I$  改变, 这一偏导结论不能直接外推。

**要素合取与交互作用。** 推导路径中形如  $K + R + C \rightsquigarrow P_i$  的表达式, 其加号代表同一工程情境下多项现实约束的逻辑合取。要素之间的数值交互通过差分效用函数  $\Delta V_i(\theta)$  中的解析关系显式刻画。例如, 若某项防护机制能将故障后的恢复负担削减固定比例  $\xi$ , 则该防护带来的期望净收益便直接包含交叉项  $p\xi I$ ; 具体的交互函数形式与物理量纲随具体情境模型显式给出。

**经验桥接机制。** 在工程实践中, 部分因果推导依赖心理认知、团队协作或特定开发工具等经验事实, 本文将其形式化表征为辅助经验前提:

$$\Psi_j = (\text{核心主张}, \text{适用范围}, \text{证据来源}, \text{效应区间}, \text{更新版本}).$$

附录中的通用  $\Psi$  标记仅提示存在经验依赖, 尚不等于完成了上述记录。进入独立检验前, 每条路径必须列明具体主张、来源、适用人群与系统、测量方法以及预期效应区间; 缺少记录的条目保留为解释提案。例如, 知识外部化能否缩短交接时间, 还取决于文档更新率、可检索性和阅读成本。不能在看到结果后才添加“团队特殊”等前提来吸收反例。

## 4 相关理论定位

本节选择直接影响模型定位与边界的研究传统进行比较。这是围绕当前论证的定向文献检索, 并非穷尽性的系统综述; 不能据此断言不存在更接近的既有框架。

### 4.1 有限理性、信息价值与推理成本

Simon 的有限理性涉及知识、计算及组织条件 [1], 信息价值理论则评价额外信息对行动收益的改善 [2]。本文的  $k_Q$  使用后者的 EVPI 定义, 只操作化了决策相关的信息缺口。它不等于人的全部认知限制, 也没有解决“计算最优策略本身太贵”的问题。Russell 与 Wefald 将计算的潜在行动收益及代价纳入元推理决策 [31]。因此, 本文的精确优化首先是规范性基准; 实际应用可以只比较少量可行方案, 以区间估计、满意解和分析预算限制建模成本。

### 4.2 软件工程经济学与实物期权

Boehm 的经济学分析与 CBAM 直接处理成本、收益和架构取舍 [19, 22]。实物期权研究强调不确定条件下保留后续选择的价值 [13]; Sullivan 等已将这一思想用于软件信息隐藏、承诺时机与分阶段开发 [21]。因而, P2 及发布案例是对这些已有机制的运用。等待外部信息与主动付费试验也应区分: 后者同时改变预算、信息和系统暴露程度。KRCIGE 的增量工作是把这些比较同任务恢复要求、实际权限和允许行动边界放在一份可检查的情境记录中。

### 4.3 面向恢复的计算与技术债务

面向恢复的计算（ROC）把快速恢复和可维护性置于系统设计的重要位置 [23]。Operator Undo 的邮件系统进一步结合状态回退、修复及用户操作重放，并处理外部可见的不一致 [24]。这些工作表明，恢复能力已有直接的软件系统研究基础；仅能部署旧二进制程序并不足以证明业务状态可恢复。本文的  $I_{\epsilon, \delta}$  提供一种依赖任务的成本表达，不能据此宣称首次发现恢复的重要性。

技术债务文献也已涉及未来变更受阻。Dagstuhl 16162 的工作定义把短期便利的设计或实现构造与后续变更成本增加、甚至变更不可行联系起来 [25]。因此，技术债务和  $I$  的范围可能重叠。本文区分的是继续维护的额外负担与指定行动后的恢复负担，不主张二者在统计或代数意义上正交。

### 4.4 控制、系统安全与组织层次

Ashby 和鲁棒控制研究为扰动、反馈与可调节范围提供基础 [3, 14]。FLP 的结论针对完全异步且可能发生进程崩溃的确定性共识模型中终止保证的限制 [7]，不能泛化为一切可靠性保证均不可能。弹性工程关注面对扰动的适应与恢复 [15]；Leveson 的 STAMP 则强调组件交互、控制约束及组织层级 [26]。由此，局部组件可靠或恢复迅速都不足以单独推出系统安全。KRCIGE 的损失函数、扰动模型和行动集合必须覆盖相关交互，单纯把未建模危险记作  $E$  并不能完成安全分析。

### 4.5 设计过程、目标形成与多维评价

Ralph 区分以规划和既定问题为中心的设计观，以及强调行动、问题重构与共同演化的设计观 [27]。这提醒本文：真实项目中的  $Q$  和  $G$  也可能随设计活动变化，不能总被视为外生常量。SPACE 则指出开发者生产力不能由单一活动指标充分衡量 [28]。因此，提交次数、生成代码量或发布频率只能作为局部观察，不能直接替代本文的目标价值。统一效用只在评价主体、权重及取舍规则已被明确接受时成立。

### 4.6 解释框架的证据要求

Gregor 对理论类型的分析，以及 Sjøberg 等的软件工程理论构建工作，支持区分概念组织、机制解释与经验检验 [29, 30]。本文需要分别回答“能否描述已有实践”“能否预先判断适用边界”“是否帮助人做出更好的决策”。这三个问题不能用同一份事后映射材料替代。后文的形式证明、合成计算、定性对应和未来评估协议据此分别报告。

## 5 理论要素与适用条件

### 5.1 K: 认识有限

**定义 5.1** (决策相关认知缺口). 在给定的工程任务  $Q$ 、决策主体  $s$ 、评估时间窗  $\tau$ 、当前观测历史  $h_t$  及剩余预算  $B$  下，令  $\mathcal{A}_Q(B)$  为同时满足预算可行性与伦理约束的候选决策空间。设  $u_Q(w, a)$  为行动  $a$  在隐状态  $w$  下所获得的条件期望净效用（对未来随机扰动按预设条件分布取

期望)。定义决策相关的认知缺口为

$$k_Q(s, \tau, h_t; B) = \mathbb{E}_{W|h_t} \left[ \sup_{a \in \mathcal{A}_Q(B)} u_Q(W, a) \right] - \sup_{a \in \mathcal{A}_Q(B)} \mathbb{E}_{W|h_t} [u_Q(W, a)]. \quad (5.1)$$

该定义基于统一的策略空间与效用度量。 $k_Q$  严密量化了获取完备任务状态信息所能带来的期望决策改善价值 (EVPI)。 $k_Q > 0$  表明在给定模型和候选决策集合中, 完备状态信息能够严格提高最优期望价值;  $k_Q = 0$  不表示主体已知全部事实, 也不排除最优行动存在并列选择。

此处状态变量  $W$  代表直接影响决策损益的任务隐状态或关键参数 (如系统接口的兼容性边界、真实业务负载分布或外部依赖版本)。 $k_Q$  确立了当前情境下信息所能蕴含的理论价值上限, 实际探针观测的净增益则由所获信号的边际改善与获取成本共同决定。在单阶段决策中  $a$  为具体的执行动作; 在序贯情境中, 候选决策则泛指所有允许继续探索或交付的后续策略分支。Simon 的有限理性学说提供了认知约束背景, 统计决策理论中的信息价值分析则为其提供了严格的量化接口 [1, 2]。

## 5.2 R: 资源有限

**定义 5.2** (资源瓶颈与影子价格). 在给定观测历史  $h_t$  下, 系统的资源有限性  $R$  由可用预算向量  $B = (B_1, \dots, B_m)$ 、行动资源消耗向量  $c(a)$  及其诱导的可行域

$$\mathcal{A}_Q(B) = \{a \in \mathcal{A}_E(h_t) : c(a) \preceq B\}$$

共同界定。在保持其他情境参数不变的基准下, 以第 2 节的最优值函数  $V_Q(h_t, B)$  为关于预算的生产效用函数, 定义资源  $j$  在有限增量  $\Delta > 0$  下的边际价值为

$$\lambda_j(\Delta) = \frac{V_Q(h_t, B + \Delta e_j) - V_Q(h_t, B)}{\Delta}. \quad (5.2)$$

其中  $e_j$  为第  $j$  类资源的分量单位基向量。当  $\lambda_j(\Delta) > 0$  时, 表明资源  $j$  在该尺度下构成制约系统效用的紧约束 (有效瓶颈); 在可微情形下, 局部影子价格收敛为连续偏导数  $\lambda_j = \partial V_Q / \partial B_j$ 。

对于离散配置资源, 可通过差分尺度  $\Delta$  进行灵敏度分析; 对于连续资源, 则通过偏导数评估其边际产出。架构选型中的技术替代 (如以存储换延迟、以算力换人力) 可通过重新配置多维预算的消耗曲线并评估值函数的变动来横向比较。实际资源的物理稀缺性、可用存量与主观折算尺度共同决定了瓶颈的严苛程度; 而系统的容灾自愈能力, 则具体体现为恢复工时、备用冗余以及跨部门协调通道等资源要素的有效投入。

## 5.3 C: 控制有限

**定义 5.3** (极小极大控制缺口). 设主体当前的后验信念为  $P(W | h_t)$ , 剩余预算为  $b_t$ , 工程预期目标集合为  $\mathcal{G}_Q$ , 目标偏离损失函数为非负的  $L_Q$ 。记  $\mathcal{D}_{t:H}$  为与当前历史相容的后续扰动序列集合,  $T_{t:H}$  为从当前状态执行策略至终期所达到的状态。先定义具体可行策略  $\pi$  的最坏期望风险, 再定义控制缺口:

$$r_Q(\pi | h_t, b_t) = \sup_{d \in \mathcal{D}_{t:H}} \mathbb{E}_{W|h_t} [L_Q(T_{t:H}(W, \pi, d), \mathcal{G}_Q)], \quad (5.3)$$

$$\gamma_Q(s, \tau, h_t; b_t) = \inf_{\pi \in \Pi_Q(h_t, b_t)} r_Q(\pi | h_t, b_t). \quad (5.4)$$

$\gamma_Q$  是可行策略最坏期望风险的下确界 (Minimax Risk); 当下确界能够取得时, 它等于最优鲁棒策略的风险。在可行策略集非空时,  $\gamma_Q > 0$  表明所有可行策略均有一个共同的正风险下界;  $\gamma_Q = 0$  仅表明风险可任意逼近零, 只有下确界可达时才能据此断言存在零风险策略。若可行策略集为空, 约定  $\gamma_Q = +\infty$ 。

若任务规定有限的风险上限  $g_{\max} \geq 0$ , 应将约束施加到实际选择的策略上:

$$\Pi_Q^g(h_t, b_t) = \{\pi \in \Pi_Q(h_t, b_t) : r_Q(\pi | h_t, b_t) \leq g_{\max}\}, \quad (5.5)$$

$$V_Q^g(h_t, b_t) = \sup_{\pi \in \Pi_Q^g(h_t, b_t)} \mathbb{E}[U_{Q,t,H}(\pi) | h_t]. \quad (5.6)$$

最终决策须从  $\Pi_Q^g$  中选择; 若该集合为空, 则当前风险上限下无可行策略, 约定  $V_Q^g = -\infty$ 。条件  $\gamma_Q \leq g_{\max}$  只是该集合非空的必要条件; 若  $\gamma_Q < g_{\max}$  或控制缺口的下确界能够取得, 则它也是充分条件。即使存在合规策略, 也不能据此认定仅按式(2.4) 的期望收益选出的策略合规。例如, 两项策略的 (期望效用,  $r_Q$ ) 分别为 (0, 0) 和 (10, 100), 且  $g_{\max} = 1$ , 此时  $\gamma_Q = 0$ , 但收益更高的第二项策略会被式(5.5) 排除。

上述约束评价当前历史之后的整段策略; 若要求每个后续可达历史都满足指定风险上限, 应将逐历史的风险要求一并纳入可行策略集。风险约束的含义由  $L_Q$  决定; 若关注中途发生的安全失效, 应使用轨迹损失, 或将相关历史信息纳入状态。

控制缺口与具体策略风险遵循鲁棒评价准则, 第 2 节的策略价值则对应给定概率模型下的期望收益, 具体工程情境可联合报告这些量。扰动集合可涵盖网络抖动、线程调度竞争、第三方服务不可用以及人为误操作; 主体所拥有的动态反馈观测与控制权限, 共同决定了策略能够调用的响应动作。在实证分析中, 可通过固定信念与预算来考察控制权限的边际影响, 或通过固定权限来探究认识缺口的作用 [3, 14]。

## 5.4 I: 工程不可逆性

设  $X \sim P(\cdot | h_t)$  为行动前状态,  $Y = T(X, a, D)$  为行动后状态,  $z$  为恢复时仍可用的证据,  $\rho \in \Pi_E$  为权限和伦理边界允许的恢复策略,  $X^\rho$  为恢复结果。任务差异函数  $D_Q(x, x') \geq 0$ 、容差  $\varepsilon \geq 0$  与允许失败概率  $\delta \in [0, 1]$  共同规定“恢复成功”的含义。差异函数不必是严格的数学度量, 但其评价对象和判定规则必须固定。

**定义 5.4** (工程不可逆性). 行动  $a$  的任务相关工程不可逆性定义为

$$I_{\varepsilon, \delta}(a | h_t, z) = \inf_{\rho \in \Pi_E} \{\mathbb{E}[C_{\text{rec}}(\rho; Y, z) | h_t, z] : \Pr[D_Q(X, X^\rho) \leq \varepsilon | h_t, z] \geq 1 - \delta\}. \quad (5.7)$$

可行恢复策略为空时, 约定  $I_{\varepsilon, \delta} = +\infty$ 。这里的成本为非负标量, 所用资源折算权重预先给定并在比较中固定; 若保留工时、资金等多维成本, 应报告可行成本集合及 Pareto 前沿, 不能直接把上式的标量下确界当作向量最优值。下确界不一定取到; 已知且满足成功约束的恢复策略, 其模型期望成本给出可达上界。有限次演练的实际成本和成功率只提供带测量误差的估计。

**恢复对象与任务边界。** 对于一次数据库迁移, 应分别检查: 旧代码能否重新运行; 数据与约束能否恢复; 故障期间的新写入能否保留或重放; 已发出的通知、付款及第三方依赖如何处理。若任务仅要求恢复旧程序, I 可以很低; 若同时要求保留新写入并完成外部补偿, 允许的恢复策略与成本就会改变。Operator Undo 对重放和外部一致性的处理提供了具体先例 [24]。

式(5.7) 的简单形式以行动前状态为参照。存在并发新业务时, 应把合法事件轨迹  $\mathcal{L}$  纳入恢复任务, 使用参照映射  $R_Q(X, \mathcal{L})$  替换比较中的  $X$ , 明确哪些新事件必须保留。历史事实无法消除不必然意味着业务目标无法恢复; 若  $Q$  允许补偿、兼容期或服务替代, 应把这些方案纳入  $\Pi_E$ 。只有规定的目标、权限和可靠性要求确实排除了所有恢复策略, 才能写  $I = +\infty$ 。涉及外部损害的补偿是否可接受由  $G$ 、 $E$  的治理过程决定。

**命题 5.5** (附加信息的事前恢复价值). 固定行动、状态与信号的联合分布、任务成功标准、成本及恢复动作。比较两种可选的信息方案  $Z_1, Z_2$ , 其中  $Z_1 = g(Z_2)$ , 允许策略忽略额外信号。令  $I_{\epsilon, \delta}^{[Z_j]}$  表示在信号揭示之前, 对状态和信号共同取期望, 并施加同一个总体成功概率约束时的最低恢复成本, 则

$$I_{\epsilon, \delta}^{[Z_2]} \leq I_{\epsilon, \delta}^{[Z_1]}.$$

证明. 任一使用  $Z_1$  的策略均可由使用  $Z_2$  的策略先计算  $g(Z_2)$  后执行。两者在联合分布下的恢复结果、期望成本和总体成功概率一致。因此细信息方案包含粗信息方案的全部可行策略, 取下确界得到结论; 该论证不要求最优值取到。□

这是事前信息方案的比较, 不是对每次具体观测值的条件成本排序。坏消息可能使某次更新后的恢复成本估计上升。若新增日志占用资源、改变系统或带来隐私成本, 其采集和保留代价还需单独进入总决策账本。

**信息擦除的一个充分条件。** 考虑离散静态恢复问题  $Y = f(X)$ 。若正概率状态  $x_1, x_2$  产生完全相同的恢复起点和全部可用证据  $(Y, z)$ , 恢复过程无法获得其他区分信号, 且二者没有共同满足零容差要求的恢复结果, 则任何基于  $(Y, z)$  的策略都无法以概率一正确恢复两者。随机化也不能同时以概率一落入两个不相交的成功集合, 故此条件下  $I_{0,0} = +\infty$ 。仅有非单射映射或观测不足还不够: 被合并的状态可能对任务等价, 系统也可能保留另一个可用于恢复的信息通道。此论证讨论任务状态的可恢复性; Landauer 关于逻辑不可逆计算与热耗散的研究 [4] 并不直接给出本文的工时或业务恢复成本。

**命题 5.6** (条件恢复成本对策略排序的贡献). 设两个行动具有相同的成功收益、故障事件概率  $p > 0$  和直接故障损失;  $I_1 < I_2$  是在该故障事件已经发生的条件下、满足同一恢复要求的最优期望恢复成本, 且最优值可达。设其行动成本为  $C_1, C_2$ , 恢复之外的结果相同, 则

$$\mathbb{E}[U(a_1) - U(a_2)] = p(I_2 - I_1) - (C_1 - C_2).$$

因此  $p(I_2 - I_1) > C_1 - C_2$  时,  $a_1$  的期望效用更高。

证明. 分别按成功与故障事件分解期望效用, 相同的收益及直接损失项抵消, 剩下前置成本差与概率加权的恢复成本差。□

此处  $I_i$  明确为故障条件下的成本; 若某项  $I$  已经对故障和成功共同取无条件期望, 就不应再次乘以  $p$ 。预防性备份和回滚路径建设属于前置成本, 实际恢复消耗在发生时计入, 二者均会影响事前策略价值。

**注 5.7** (工程不可逆性与技术债务的概念区分). 技术债务描述设计或实现选择给后续维护和变更带来的负担 [16, 25];  $I$  则回答某次行动之后、按指定成功标准恢复需要多少成本。两者可以相关, 也可能指向同一项结构问题。为避免重复计费, 可以将某条给定演化路径上的额外维护成本记作  $C_{\text{debt}}$ , 将恢复操作消耗单独列账, 并明确共同成本的归属。



一个仍在维护、修改成本很高的遗留模块，若此次改动隔离良好且数据快照完备，仍可能低成本回退；反之，一个维护负担较低的公共 API，在形成外部依赖后可能需要昂贵的兼容迁移。两个例子说明两项问题值得分别询问，不能证明统计独立、严格正交或任意极值组合的存在。一次性脚本的代码异味不自动构成高额未来债务，形式化验证也不证明公共 API 的技术债务为零。

## 5.5 G: 工程目标

**定义 5.8** (工程目标). 工程目标  $G$  给出对系统状态和行动的偏好。本文采用宽泛表述：

$G$ : 在约束下持续、可靠地交付预期价值。

$G$  确立了系统优化的价值导向。K、R、C、I 客观刻画了物理世界与人类认知所固有的客观有界性，而  $G$  则明确了决策主体在这些约束可行域内所追求的最优效用方向。

## 5.6 E: 伦理边界

**定义 5.9** (伦理边界). 伦理边界  $E$  为每个决策历史定义允许行动集合

$$\mathcal{A}_E(h_t) \subseteq \mathcal{A}_s.$$

工程策略的所有行动必须满足硬性约束

$$a_t \in \mathcal{A}_E(h_t).$$

该要素用于形式化表达系统安全红线、用户隐私权、法律合规、社会责任以及防范重大危害等非可协商约束。工程目标  $G$  可以在连续可行域内进行多指标帕累托权衡，而伦理边界  $E$  则充当了不可逾越的刚性边界。

## 5.7 目标、风险承担与跨层治理

单主体值函数要求目标和约束已被明确。真实项目中，决策者、受益者、受损者和恢复执行者可能不是同一群体。对同一候选策略，应先分别记录用户、开发团队、运维团队、合作方及未来维护者承担的收益、损失和资源需求；存在未被内部预算覆盖的外部成本时，局部最优不保证整体可接受。

$G$  的确定至少需说明谁参与定义目标、谁接受权重及折现选择、谁承担剩余风险。若各方尚未接受一个共同标量目标，可保留效用向量  $(U_1, \dots, U_m)$ 、最低保障条件和 Pareto 比较；权重协商本身属于治理问题，不能由 K、R、C、I 自动推出。 $E$  则需要相应的约束来源、检查证据、执行责任与更新机制；把它写成集合并不等于现实中已经能准确识别或强制执行。

时间边界同样重要。短期团队可能省下前置投入，却把数据修复、兼容支持或知识重建的成本交给下一任维护者。评估时应公开终止残值  $v_H$  中包含的义务，并检查相邻视界下策略排序是否稳定。组织层的权限和激励也会改变项目层的  $\mathcal{A}_s$ 、预算与证据可得性，这与系统安全中的分层控制视角相容 [26]。本文未建立完整多主体均衡或组织行为模型；这些问题是使用单主体模型前必须显式处理的边界。

## 5.8 $\Sigma$ : 可学习结构条件

**定义 5.10** (可学习结构条件). 给定参考环境集合  $\mathcal{E}_Q$ , 在其中一个已声明的环境模型  $e$  下, 令  $Z$  为观测、 $h_t$  为当前历史、 $B$  为当前可用预算, 定义观测的净决策价值

$$\text{NVOI}_{Q,e}(Z | h_t) = \mathbb{E}_{Z|h_t,e} \left[ \sup_{a \in \mathcal{A}_Q(B - c_Z)} \mathbb{E}[U_Q | Z, h_t, e, a] \right] - \sup_{a \in \mathcal{A}_Q(B)} \mathbb{E}[U_Q | h_t, e, a] - C_Q(Z). \quad (5.8)$$

$C_Q(Z)$  记录观测的资源折算、延迟及试验暴露代价;  $c_Z$  仅用于更新剩余预算, 不重复扣减效用中的同一笔成本。

本文把  $\Sigma$  用作反馈适用结构的声明, 要求在观察结果出现前说明: 信号如何对应待判断的状态或结果; 试点与目标场景在哪个时间窗内可迁移; 选择偏差、干预效应与环境漂移如何检测。它不是“凡是成功学习就满足”的事后标签, 也不声称给出了统计学习理论中的通用可学习性判据。应通过预留时段或环境的数据检查这些声明; 无法识别信号来源或迁移范围时, 净价值估计需保留不确定性。

信号与目标相关, 例如互信息  $\mathcal{I}_{\text{Shannon}}(Z; Y) > 0$ , 不保证值得采集: 它可能不改变任何可行决策, 或其成本超过收益。 $\Sigma$  说明何时可以依据某种反馈模型进行推断;  $\text{NVOI} > 0$  则是在该模型和预算下应否投资于这项观测的决策条件。二者分开后, 即使环境可学习, 也可能合理地停止试验。No Free Lunch 的有限搜索结果提醒算法比较依赖问题类与平均方式 [8], 但不直接证明某个工程信号具备上述迁移条件。

## 6 中间原则

为减少从 KRCIGE 到具体实践的重复解释, 本文组织十三条中间原则。下文的“原则”是机制性决策规则; 只有给出具体成本、效果与可行性前提时, 才产生数学排序。实践名称本身不保证这些前提成立。

### 6.1 P1: 反馈与信息价值

**派生原则 6.1** (反馈价值原则). 在观测模型适用、行动可行且价值账本一致时, 若一项观测的净决策价值为正, 则“取得观测后再选择”的策略优于所比较的无观测基线。多项观测竞争同一预算时, 还需比较它们的净价值和相互依赖。

在统一模型中, 观测的净决策价值由式(5.8) 给出。当

$$\text{NVOI}_{Q,e}(Z | h_t) > 0$$

时, 观测具有正工程价值。

原型、自动测试、性能测量及用户反馈可以提供此类观测, 但需要分别证明信号有效且足以改变后续行动。若两种可能结果都导致同一选择, 或实验用户不能代表目标用户, 则信息相关性本身不足以支持投入。

### 6.2 P2: 选择权与最少承诺

**派生原则 6.2** (选择权原则). 在后续信息能改变选择、延迟不消除候选方案且总成本可控时, 保留选择权可能提高期望价值。其优势取决于未来可选策略的改善幅度与等待、维护选择权的代

价。

相对于当前承诺策略，等待的净价值可记为

$$\Delta V_{\text{wait}} = V_{\text{later choice}} - V_{\text{commit now}} - C_{\text{delay}} - C_{\text{preserve}}.$$

各价值项使用同一时点和收益尺度，且右侧末两项尚未包含在前两项中。只有  $\Delta V_{\text{wait}} > 0$  才支持延迟。机会窗口关闭、兼容层持续维护或需求不确定性不可通过等待降低时，立即承诺可能更好；不确定性增加也不能脱离收益分布与成本结构推出统一方向 [21]。

SICP 将数据抽象解释为最少承诺原则的应用：抽象屏障允许具体表示选择延后，从而保留系统灵活性 [6]。

### 6.3 P3: 复杂度预算

**派生原则 6.3** (复杂度预算原则). 复杂度消耗有限资源。新增复杂度应带来足以覆盖实现、理解、运行和变更成本的工程价值。

可以使用全生命周期总成本表达：

$$C_{\text{total}} = C_{\text{implementation}} + C_{\text{understanding}} + C_{\text{operation}} + C_{\text{change}}.$$

该原则为简单设计（KISS）、算法渐近复杂度分析、剔除无收益抽象、扁平化继承层次以及避免追逐无实质收益的技术重写提供了严格的经济学依据。

### 6.4 P4: 局部化与模块化

**派生原则 6.4** (局部化原则). 在  $K + R + C + G$  条件下，系统设计应尽可能缩小单次变更所牵涉的认知理解、团队协调与并发控制范围。

可以将模块化划分目标形式化为全生命周期成本极小化问题：

$$\min (\mathbb{E}[C_{\text{propagation}}] + C_{\text{boundary}}).$$

其中第一项代表变更的跨模块级联传播成本，第二项代表接口契约、独立部署、进程通信及边界协调的治理成本。

Parnas 的开创性工作将模块划分与系统的可变性及可理解性紧密结合 [5]。上述目标函数同时为评估单体模块化边界、面向服务架构（SOA）及微服务拆分粒度提供了统一的量化权衡框架。

### 6.5 P5: 显式假设

**派生原则 6.5** (显式假设原则). 在  $K + C + I$  条件下，凡跨越模块边界、协作团队或演化时间周期的关键业务假设，均应尽可能编码为显式、机器可验证的数据模式、强类型约束、前置/后置契约或自动化测试用例。

该原则直接衍生出契约式设计（Design by Contract）、断言检查、静态类型系统、Schema 校验、值对象显式状态机、基于属性的测试（Property-based Testing）以及形式化可执行规范。

## 6.6 P6: 信息保留与证据价值

**派生原则 6.6** (信息保留原则). 在满足伦理与合规边界的前提下, 若某项上下文信息在未来被调用的概率为  $p$ , 其丢失后引发的额外决策与恢复损失为  $L$ , 持久化存储成本为  $S$ , 而保留该信息可能诱发的数据安全性与隐私暴露风险折算为  $Q$ , 则当且仅当

$$pL > S + Q$$

时, 持久化该项信息具备正向期望收益。

该原则为版本控制全量历史、结构化日志、事件审计轨迹、原始请求上下文与高精度中间语义表示提供了定量正当性。凡触犯合规红线的持久化行为首先被移出可行域, 其余数据策略则在此不等式下权衡未来决策价值、恢复保障、存储开销与外溢风险。

## 6.7 P7: 权威来源与协调语义

**派生原则 6.7** (权威来源原则). 当系统中同一业务事实存在多个数据副本或派生表达时, 必须显式确立单一权威来源 (Single Source of Truth) 或形式化定义冲突仲裁与协调语义。

其推导源自多重约束的联合作用:

$$R + C + I + G.$$

跨副本数据同步必然消耗通信与计算资源; 外部并发写入与部分失效急剧放大分叉概率; 而一旦发生分叉, 重新确认主次事实将面临高额的信息恢复与状态协调代价。

该原则为 DRY 原则 (Don't Repeat Yourself) 给出了更为深层且广义的现代解释:

同一权威知识需要明确 authority 或 reconciliation。

## 6.8 P8: 自动化阈值

**派生原则 6.8** (自动化原则). 设人工流程执行  $n$  次的预期成本为

$$nC_h + R_h,$$

自动化方案成本为

$$F + nC_a + R_a.$$

当

$$F + nC_a + R_a < nC_h + R_h$$

时, 自动化具有正期望收益。

这里  $F$  为自动化的前期固定研发成本,  $C_h, C_a$  分别为人工作业与自动化单次运行的边际成本,  $R_h, R_a$  则为二者在执行变异与操作失误下的潜在风险期望损失。

## 6.9 P9: 状态空间控制

**派生原则 6.9** (状态空间原则). 在  $K + R + C$  条件下, 软件架构设计应主动约束和剪裁系统需要推理、测试与并发协调的可达状态空间。

无约束的共享可变状态会诱发并发执行交错 (interleaving) 组合数的爆炸。消息传递、不可变数据结构、显式事务边界、Actor 模型与受限有限状态机, 本质上均属于状态空间剪裁与控制技术。

该原则亦兼容受控的状态共享。数据库事务隔离级别、分布式锁、共识协议与显式因果一致性模型, 均可作为提供精确协调语义的有效手段。

## 6.10 P10: 不可逆性与恢复能力

**派生原则 6.10** (可恢复性原则). 行动的工程不可逆性越高, 预防错误和降低恢复负担的机制通常具有越高价值。

若防护前后的故障概率分别为  $p_0, p_1$ , 故障条件下的直接损失与恢复成本为  $(L_0, I_0), (L_1, I_1)$ , 防护成本为  $C_{\text{guard}}$ , 其他期望后果变化为  $C_{\text{side}}$ , 则在其余收益相同时,

$$\Delta V_{\text{guard}} = p_0(L_0 + I_0) - p_1(L_1 + I_1) - C_{\text{guard}} - C_{\text{side}}.$$

当该差值为正且防护策略满足预算和边界要求时, 投入才具有相对优势。固定  $p > 0$  且防护降低恢复成本的固定比例  $\xi > 0$  时, 恢复收益项为  $p\xi I$ , 随  $I$  上升; 若备份无法恢复当前数据、回滚破坏新写入或防护本身增加故障, 则不能沿用这一单调结论。

生产迁移、权限变更和外部发布应分别检查预防、故障限制、恢复与补偿的作用范围。若任务约束禁止某项后果, 仅有正期望净收益不能使行动变得可行。

## 6.11 P11: 安全暴露面与爆炸半径

**派生原则 6.11** (安全约束原则). 在  $R + C + I + E$  条件下, 系统架构应主动最小化外部恶意实体可操纵的攻击暴露面, 并严格隔离单次安全失效或异常扩散所能造成的最大爆炸半径。

该原则推导出最小特权原则 (PoLP)、缩减暴露端口与接口、安全漏洞热补丁响应、故障域隔离舱壁以及最小化收集数据等现代安全实践。

## 6.12 P12: 知识外部化

**派生原则 6.12** (知识外部化原则). 关键的系统工程知识应脱离对个体人脑生物记忆的单一依赖, 建立机器可读或团队共享的持久化外部载体。

推导路径为

$$\begin{aligned} K + R + I + G &\rightsquigarrow \text{个人认知容量有限且随时间衰减,} \\ &\rightsquigarrow \text{文档、测试用例、代码契约与版本历史.} \end{aligned}$$

知识外部化的收益还依赖内容准确、持续更新和可检索; 过时文档可能误导后续主体。保存、审查与阅读成本必须进入比较, 知识保留也受隐私和权限边界约束。

## 6.13 P13: 模型持续校正

**派生原则 6.13** (模型更新原则). 工程计划、需求规格、架构设计及成本估算, 本质上均属于主体关于外部世界的信念模型。当在演化过程中捕获到新的经验证据时, 应权衡证据的信息信噪比与重构成本, 及时修正既有模型。

形式化为

$$M_t \xrightarrow{O_{t+1}} M_{t+1}.$$

该原则由

$$K + C + \Sigma + G$$

驱动，并直接对应敏捷滚动计划、持续需求演进、性能基准校准以及基于实际产出的自适应技术选型。

## 7 可检验假设与机制评价

为使原则体系具备可沉淀、可检验且可证伪的科学属性，本文将每条中间原则规范化表征为如下形式化假设四元组：

$$\mathcal{H}_i = (\text{前提}, \text{机制}, \text{指标}, \text{方向与边界}).$$

其中，前提记录具体情境及其经验依赖，机制说明因素如何影响策略价值，指标指定可观察的测量方式，方向与边界给出待检验预测。不同项目的代理指标可能存在测量差异，需要另行校准。候选假设汇总见附录 C（表 3）。

### 7.1 机制分组与消融评价

十三条原则可按功能分为四组：信息与学习  $\{P_1, P_5, P_6, P_{13}\}$ 、选择权与恢复  $\{P_2, P_{10}\}$ 、复杂度与局部化  $\{P_3, P_4, P_9\}$ ，以及权威与执行  $\{P_7, P_8, P_{11}, P_{12}\}$ 。这是一种阅读组织方式。语料条目数  $N$  除以分组数  $M$ ，即  $\rho = N/M$ ，只反映条目与分组的数量关系；任意合并分组就能提高该数值，故不作为泛化能力或理论压缩效果的证据。

若要评估解释的简洁性，应计入定义、桥接前提、映射规则以及未解释例外的复杂度。最小描述长度方法同时考虑模型与数据编码，为此提供了方法论参照 [32]；本稿未规定可计算的编码方案，也未执行 MDL 模型比较。

未来消融可以使用

$$\Delta_i = \text{Score}(\mathcal{H}) - \text{Score}(\mathcal{H} \setminus \{P_i\}).$$

评分规则、测试材料和基线在评估前固定，移除  $P_i$  后不得临时让其他原则吸收其含义。预测损失、边界判断和决策结果应分别报告；若汇总成单一分数，需公开权重。由于原则可能互补或冗余，单项  $\Delta_i$  还应结合成组消融和不确定区间解释。本文尚无此类消融结果。

## 8 代表性实践推导

### 8.1 DRY

设同一业务知识  $q$  被复制到  $n$  个位置：

$$q_1, q_2, \dots, q_n.$$

每次规则变化需要同步更新多个副本，产生维护成本：

$$C_{\text{update}} \propto n.$$

在控制有限性  $C$  作用下，跨副本的原子更新需要协调协议，并受到通信条件、节点失效与终止性要求的约束；在工程不可逆性  $I$  约束下，副本分叉后必须付出重新判定主次事实、解决内容冲突及修复一致性的高额成本；在资源有限性  $R$  限制下，依赖持续的人工协调来维持多副本同步在经济上不可行。

因此：

$$R + C + I + G \rightsquigarrow P7 \rightsquigarrow \text{DRY}.$$

这一推导将 DRY 的实质约束对象精确锚定于“权威知识”而非表层代码。当两个代码片段虽然文本表象相似、但在业务领域中承载完全独立的语义时，强行合并抽象反而会破坏选择权，此时保持正交演化更具工程优越性。

## 8.2 YAGNI 与避免预言未来

未来需求属于当前未知：

K.

提前实现未来功能立即消耗资源：

R.

远期业务演化的真实形态具有高度不确定性 ( $K$ )；过度提前实现未经验证的功能会直接挤占当前紧缺的开发与测试资源 ( $R$ )；过早引入多余的抽象层、数据模式或外部依赖，不仅会缩减未来的策略空间，还会大幅抬高未来推翻该设计时的回滚门槛 ( $I$ )：

I.

因此，在需求尚未获得足够证据时：

$$K + R + I + G \rightsquigarrow P2 + P3 \rightsquigarrow \text{延迟未经证实的结构承诺}.$$

## 8.3 版本控制

代码演化处于持续的扰动与控制缺口之下 ( $C$ )：误改、逻辑缺陷、依赖破坏及团队并发冲突在所难免。若采用原位覆盖 (In-place overwrite) 而缺少持久化的历史证据  $z$ ，代码变更的不可逆性  $I_{\epsilon,\delta}$  将陡增至不可控水平；而依赖手动归档又面临不可持续的工时开销 ( $R$ )。

因此：

$$C + I + R + G \rightsquigarrow P6 + P10 + P8 \rightsquigarrow \text{版本控制}.$$

版本控制系统将全量修改历史固化为可寻址的可用证据集合  $z$ ，以极低的边际成本提供了确定性的回退路径，从而系统性压低了软件变更的工程不可逆性。

## 8.4 自动化测试

当前代码实现的正确性受限于主体的先验认知 ( $K$ )；未来运行时的负载输入与依赖环境充满动态不确定性 ( $C$ )；而依靠人工回归验证则面临沉重的时间与人力成本 ( $R$ )。

在可学习结构条件  $\Sigma$  满足的前提下，测试执行信号对排查潜在故障具备显著的净信息价值：

$$K + C + R + \Sigma + G \rightsquigarrow P1 + P5 + P8 \rightsquigarrow \text{自动化测试}.$$

测试还能把已发现的行为约束与缺陷现场形式化外部化:

$$P12.$$

因此“每个 Bug 只需被修复一次”可进一步表示为

$$\text{Bug 证据} \rightsquigarrow \text{回归测试} \rightsquigarrow \text{长期保存新获得的不变量知识.}$$

## 8.5 低耦合

若某模块的局部调整会强行迫使大量关联模块同步修改, 则理解单次变更所需的认知上下文、跨组协作成本以及故障连锁传播的风险均将急剧放大。

因此:

$$K + R + C + G \rightsquigarrow P4 \rightsquigarrow \text{低耦合、封装与局部化.}$$

进一步加入接口契约维护与通信边界管理成本, 可获得显式的条件化优化问题:

$$\min (C_{\text{change propagation}} + C_{\text{boundary}}).$$

该目标函数能够统一解释从单体模块化重构到分布式微服务边界划分的权衡过程。

## 8.6 显式时间语义

设系统需要跨进程传递一个确定时刻及其业务时区语义:

$$x = (t, z).$$

若在传输中进行有损序列化:

$$f(t, z) = t_{\text{local}},$$

多个不同的  $(t, z)$  将映射至完全相同的局部字符串。I 表明在缺乏外部附加坡载证据时, 接收端在数学上无法无损逆向恢复原始的时间语义; K 则表明调用方无法确切推断被调用方隐式假定了何种时区基准。

因此:

$$K + I \rightsquigarrow P5 + P6 \rightsquigarrow \text{显式携带 Instant、offset、zone 或明确的时间语义.}$$

# 9 案例研究：直接发布与分阶段发布

本节首先建立静态单阶段决策基准, 进而拓展至同一 Beta-Binomial 证据更新机制下的有限时域序贯动态规划模型。案例中所有参数均为完全公开可复算的标定输入, 旨在定量展示策略选择边界、预算约束收紧以及贝叶斯模型更新的相互作用机制。



## 9.1 静态单阶段基准

设团队准备发布一项会改变外部客户端交互协议的高危功能。先验缺陷概率设定为共轭先验分布：

$$q \sim \text{Beta}(1, 9).$$

在 40 个集成测试场景中观测到 3 个缺陷与 37 次成功执行，后验更新为

$$q \mid o \sim \text{Beta}(4, 46), \quad \bar{q} = \mathbb{E}[q \mid o] = 0.08.$$

对比直接全量发布  $a_D$  与基于特性开关的分阶段金丝雀发布  $a_S$ 。价值与代价均统一度量等价为工程工时（Engineering Hours, EH）：

| 变量                        | 直接发布 $a_D$ | 分阶段发布 $a_S$ |
|---------------------------|------------|-------------|
| 成功交付价值 $V$                | 240        | 240         |
| 实现与发布成本 $C$               | 20         | 35          |
| 延迟价值损失 $C_{\text{delay}}$ | 0          | 12          |
| 缺陷发生后的直接损失 $L$            | 600        | 60          |
| 工程不可逆性 $I_{0,0.01}$       | 80         | 6           |

直接发布和分阶段发布的期望效用分别为

$$\mathbb{E}[U_D] = 0.92 \times 240 - 20 - 0.08(600 + 80) = 146.40,$$

$$\mathbb{E}[U_S] = 0.92 \times 240 - 35 - 12 - 0.08(60 + 6) = 168.52.$$

在当前参数设定下，分阶段发布的期望净收益优势为  $\mathbb{E}[U_S] - \mathbb{E}[U_D] = 22.12$ 。两项策略在相同的先验缺陷率、价值基准与恢复成本定义下进行对比； $L$  记录线上事故造成的直接经济损失， $I$  衡量系统回滚与状态恢复所必须消耗的额外资源，各项代价在工程价值账本中实现独立精确核算。

两类策略的静态效用差对缺陷率  $q$  的敏感性函数为

$$\mathbb{E}[U_S - U_D] = -27 + 614q.$$

由此导出分阶段发布的策略选择临界阈值为

$$q > \frac{27}{614} \approx 0.0440.$$

当前置可用预算低于 35 EH 时，分阶段发布方案  $a_S$  因突破资源约束而被移出可行策略集；当环境稳定性条件  $\Sigma$  出现漂移时，需重新测算潜在直接损失  $L_S$ ；当伦理与安全红线收紧时，则需重新标定可用证据空间  $z$  与允许的恢复策略集  $\Pi_E$ 。

## 9.2 有限时域序贯递推

令一次试点观察  $X \in \{0, \dots, m\}$  个缺陷，满足

$$X \mid q \sim \text{Binomial}(m, q), \quad q \mid o \sim \text{Beta}(\alpha, \beta).$$

观察  $X = x$  后,

$$q \mid o, x \sim \text{Beta}(\alpha + x, \beta + m - x), \quad \bar{q}_x = \frac{\alpha + x}{\alpha + \beta + m}.$$

试点后的决策集合定义为  $\mathcal{B} = \{e, r, o\}$ , 分别表示扩大、回滚和继续观察。给定剩余试点轮数  $n$ 、预算  $b$ 、后验参数  $(\alpha, \beta)$  和活动状态  $z \in \{0, 1\}$ , 定义

$$V_n(\alpha, \beta, b, z) = \max_{a \in \mathcal{B}_n} \mathbb{E}[U(a) \mid \alpha, \beta, b, z]. \quad (9.1)$$

终止扩大和回滚的价值为

$$V_e(\alpha, \beta) = (1 - \bar{q})V - C_e - \bar{q}(L_e + I_e), \quad V_r = -C_r. \quad (9.2)$$

继续观察的价值为

$$V_o = -C_o - C_{\text{delay}} + \sum_{x=0}^m \Pr(x \mid \alpha, \beta) [-\ell x + V_{n-1}(\alpha + x, \beta + m - x, b - C_o, 1)], \quad (9.3)$$

其中

$$\Pr(x \mid \alpha, \beta) = \binom{m}{x} \frac{B(\alpha + x, \beta + m - x)}{B(\alpha, \beta)}. \quad (9.4)$$

当  $n = 0$  时,  $\mathcal{B}_n$  只包含当前可行的终止行动; 当  $z = 0$  时, 保留停止行动  $V_{\text{stop}} = 0$ 。最优策略为

$$a_n^*(\alpha, \beta, b, z) = \arg \max_{a \in \mathcal{B}_n} \mathbb{E}[U(a) \mid \alpha, \beta, b, z].$$

本例中, 扩大行动要求  $b \geq C_e$  且  $B_{\text{rec}} \geq I_e$ ; 活动试点的回滚要求  $b \geq C_r$ ; 观察要求  $n > 0$  且  $b \geq C_o + C_r$ , 以保留终止退路。仅在试点尚未启动时允许零成本停止。递推求得的是给定先验、似然、动作集、成本和预算下的最优策略, 不能外推为现实项目的全局最优。

本例的  $I_e$  是扩大失败条件下的恢复成本标量, 并假定该恢复可达到规定成功标准; 因此期望损失中乘以  $\bar{q}$ , 预算筛选则保留完整恢复额度。若恢复成本具有分布或成功率未知, 需扩展模型, 不能用其均值直接保证资源充足。

### 9.3 可复现的数值求解

附带脚本 `examples/sequential_release.py` 使用合成参数

$$(\alpha, \beta, m, n) = (4, 46, 10, 2), \quad (V, C_e, L_e, I_e, C_r) = (240, 20, 2400, 80, 6),$$

$$(C_o, C_{\text{delay}}, \ell, B_0, B_{\text{rec}}) = (2, 1, 3, 30, 100).$$

观测信号采用可交换的 Beta-Binomial 共轭模型; 灰度探测中单次调用失败的直接代价为  $\ell x$ , 应急恢复预算则单独列账管控。本例在上述预算约束下优化期望效用。如需施加式(5.5)的最坏期望风险上限, 还应指定扰动集合、风险损失函数与阈值, 并对候选策略进行相应筛选; 恢复预算约束本身不能替代策略级风险约束。精确后向递推得到

$$V_2(4, 46, 30, 0) = 12.314852, \quad V_1(4, 46, 30, 0) = 9.393790,$$

而根节点三个可行行动的价值为

$$V_{\text{stop}} = 0, \quad V_e = 2.400000, \quad V_o = 12.314852.$$

因此根节点选择继续观察。第一轮观察后的策略为： $x = 0$  时扩大， $x = 1$  时继续观察， $x \geq 2$  时回滚；相应的预测概率为

$$\Pr(x = 0) = 0.465533, \quad \Pr(x = 1) = 0.338569, \quad \Pr(x \geq 2) = 0.195898.$$

附带脚本检查预测概率归一化、后验均值鞅性质，以及显式展开的两轮树与动态规划值的一致性。每组场景使用独立参数快照和缓存，避免改变参数后误用旧结果；树展开也检查预算不足和扩大不可行的分支。所有数据均为合成输入。

#### 9.4 参数敏感性与基线比较

脚本参数 `--sensitivity` 输出 11 组场景。每行均比较在相同输入下禁止观察、最多观察一轮和最多观察两轮的最优值；除“变更参数”一列外，其余参数与前述基准相同。它们是限制后续选择的策略基线，不是不同预测模型。结果单位均为前文统一的工程价值单位。

| 变更参数                               | 无观察     | 一轮      | 两轮      | 根行动 |
|------------------------------------|---------|---------|---------|-----|
| 基准                                 | 2.400   | 9.394   | 12.315  | 观察  |
| $L_e = 600$                        | 146.400 | 146.400 | 146.400 | 扩大  |
| $C_o = 10$                         | 2.400   | 2.400   | 2.400   | 扩大  |
| $(\alpha, \beta) = (8, 42)$        | 0.000   | 0.000   | 0.000   | 停止  |
| $B_{\text{rec}} = 79$              | 0.000   | 0.000   | 0.000   | 停止  |
| $B_0 = 19$                         | 0.000   | 0.000   | 0.000   | 停止  |
| $m = 3$                            | 2.400   | 6.494   | 7.103   | 观察  |
| $B_{\text{rec}} = 1000, I_e = 0$   | 8.800   | 13.908  | 15.660  | 观察  |
| $B_{\text{rec}} = 1000, I_e = 80$  | 2.400   | 9.394   | 12.315  | 观察  |
| $B_{\text{rec}} = 1000, I_e = 400$ | 0.000   | 0.000   | 0.423   | 观察  |
| $B_{\text{rec}} = 1000, I_e = 800$ | 0.000   | 0.000   | 0.000   | 停止  |

直接故障损失较低时，试验收益不足以覆盖观察和暴露成本；观察成本升高时，还会占用后续行动预算。恢复预算不足则直接排除扩大行动，信息不能单独解除这一可行性约束。提高先验故障率的场景维持  $\alpha + \beta = 50$ ，但同时改变先验均值和方差，因此仅说明这一具体先验变更的结果。

最后四行始终保持  $B_{\text{rec}} = 1000$ ，用于区分恢复成本变化与预算门槛。随  $I_e$  上升，两轮最优值递减；但观察相对无观察的增量价值依次约为 6.860, 9.915, 0.423, 0，并不单调上升。风险过高时，可利用的信息已经不能带来值得执行的后续策略。这是 P1、P2 需要明确收益与可行性前提的具体例子。

最多两轮的策略空间包含最多一轮及零轮的策略，因此其最优值不降低是允许忽略额外机会的既有决策性质，不构成 KRCIGE 优于贝叶斯决策方法的证据。输入相同且求解器相同时，任何框架都应得到相同数值。真正有待检验的增量是：KRCIGE 是否帮助团队发现遗漏的恢复任务、外部损失或可行方案，并以合理建模成本形成更好的输入。

## 10 讨论

### 10.1 原则冲突与条件化推导

KRCIGE 理论框架的一个重要价值，在于从形式化约束视角剖析并化解经典经验原则之间的内在张力。工程实践中看似相互对立的格言（例如 DRY 与推迟抽象、Fail Fast 与优雅降级），本质上均源于不同约束条件在特定系统层级或生命周期阶段的相对权重变化；形式化建模的目标正是界定多个边界约束共同作用下的最优策略区间。

#### 10.1.1 DRY 与抽象时机

DRY 原则（P7）旨在消除事实与逻辑的冗余表达以保障一致性；而控制抽象时机的决策准则（P2）则主张推迟不可逆的结构承诺以保留未来演进的选择权。

在系统演化初期，若多个实现片段尚缺乏充分证据证明其承载相同的业务本质知识（表现为决策认知缺口  $k_Q$  较大），过早抽象会引入虚假耦合，此时保留设计灵活性的期权价值更高；随着业务复用场景的积累与共同变化模式的明确确立，提炼单一权威知识源（P7）所带来的维护与一致性收益逐步占据主导。

由此推导出条件化权衡准则：

先观察共同变化，再抽取共同权威知识。

#### 10.1.2 Fail Fast 与 Graceful Degradation

在系统架构内部靠近故障根源的执行路径上，若允许受损状态隐式继续传播，将迅速丢失第一现场的上下文诊断信息并加剧不可逆破坏，因此契约校验与证据保留原则（P5 + P6）支持立即中断并快速暴露错误（Fail Fast）。

而在靠近终端用户或核心商业价值交付的外部系统边界处，未捕获异常导致的单点崩溃极易引发级联雪崩并造成不成比例的业务损失，因此系统韧性与安全原则（P10 + P11）支持实施异常隔离、服务降级与容错兜底（Graceful Degradation）。

由此导出系统纵深防御的分层准则：

局部尽早暴露错误，系统边界限制故障传播。

#### 10.1.3 小步迭代与原子不变量

选择权原则（P2）主张缩小单次交付与决策的承诺规模，而可恢复性原则（P10）要求每项变更均具备低成本可回退性。然而，当业务系统存在强一致性约束或跨组件原子不变量时，将变更机械拆解为互不一致的微小碎片反而会破坏系统整体的合法性。在此类场景下，安全变更的最小粒度并非由物理代码行数决定，而是由维持关键系统不变量所需的最小原子闭包决定。

由此导出变更粒度准则：

选择能够独立保持关键不变量的最小可恢复步骤。

#### 10.1.4 配置化与状态空间

通过外部配置解耦运行策略，能够延后系统的决策绑定时间并保留策略选择权（P2）；但与此同时，每一个新增的外部配置参数与运行时开关，都会使状态空间（P9）与潜在交错路径呈组合爆炸式增长，推高测试、排错与心智负担。

由此导出系统正交设计准则：

高变化频率策略适合显式配置；高稳定度不变量适合固化在类型、schema 与代码约束中。

#### 10.1.5 Postel 鲁棒性原则

Postel 鲁棒性原则（“在发送时严谨，在接收时宽容”）主张通信协议接收方包容非规范输入，以吸收外部异构实现引发的控制边界差异（C）；然而，过度宽容会掩盖发送方的缺陷实现，阻断关键的错误诊断反馈，并导致语义歧义在协议生态中长期固化并形成生态死锁。RFC 9413 对鲁棒性原则在互操作性演进中的长期危害进行了系统反思 [12]。

在 KRCIGE 框架下，这一张力被清晰刻画为

$$C \text{ 与 } K+I$$

之间的本质权衡：在缺乏集中协调机制的遗留生态中，宽松容忍具有吸收不可控外部实现的现实工程价值；而对于具备主动协调能力的现代协同系统，严格的契约检验能够持续提供高价值诊断反馈，并显著压缩协议状态空间的漂移。

### 10.2 进一步原则

以下原则虽未直接收录于特定经验格言清单中，但完全由 KRCIGE 的约束体系与信息推导链条自然衍生，旨在展示本框架在导出新颖工程设计准则方面的理论生成能力。

#### 10.2.1 证据半衰期原则

在软件系统的动态运行过程中，大量用于异常诊断、审计与系统恢复的关键信息具有瞬态易逝性（例如高并发请求的原始载荷 Payload、多线程竞态调度的微观时序、下游三方依赖的瞬时异常响应以及内存崩溃核心转储现场）。若未能及时捕获并持久化，这些信息将随进程退出、状态覆盖或垃圾回收而永久灭失。设时刻  $t$  可用于系统恢复的证据集为  $z_t$ ；在缺乏持续采集机制且证据集合随时间推移而单调收缩的条件下，通常有

$$z_{t+1} \subseteq z_t \rightsquigarrow I_{\varepsilon, \delta}(a \mid h_t, z_{t+1}) \geq I_{\varepsilon, \delta}(a \mid h_t, z_t).$$

定义证据的决策与诊断价值为

$$V_e(t)$$

且随时间推移发生单调衰减。若其衰减斜率满足

$$\frac{dV_e}{dt} \ll 0,$$

则推迟采集将导致不可逆性与排错成本急剧攀升，必须将证据捕获动作推向最靠近事件发生的第一现场。

由此导出：

越难重现的证据，越应靠近事件源自动捕获。

### 10.2.2 保证与前提假设

计算系统中的任何高等级可靠性保证，均非无条件的绝对真理，而是严格建立在对运行环境的一组先验边界假设之上。记系统对外承诺的工程保证为  $G_u$ ，其依赖的前提假设集合记为

$$\mathcal{H} = \{H_1, \dots, H_n\}.$$

架构评审与系统契约应显式记录每项保证依赖的前提。将实现满足相应协议或契约要求一并纳入前提后，经证明或验证的保证可表示为：

$$\left( \bigwedge_{i=1}^n H_i \right) \implies G_u.$$

该蕴含表达一组足以支持保证的前提，不要求保证与这组前提互为充要条件；不同的前提组合可能支持同一保证。

分布式共识领域的 FLP 不可能性定理为此提供了经典佐证：在完全异步的消息传递模型中，即使消息可靠传递且至多有一个进程崩溃，任何满足一致性与有效性要求的确定性共识协议，都存在一个允许的执行使协议无法终止 [7]。该结论限制的是在所有允许执行中保证终止的能力，并不否定协议维持一致性安全性质的可能性。讨论工程实现的进展保证时，必须同时说明其采用的时序与故障假设。

### 10.2.3 副本权威原则

为提升读取吞吐量、降低网络延迟或保障局部可用性，系统架构允许数据存在多个物理冗余副本；然而，副本复制必然伴随着控制分散与状态分叉风险。为此，架构设计要求任何多副本系统必须显式确立事实主权或一致性收敛语义：

replication  $\rightsquigarrow$  authority 或 reconciliation semantics.

无论是应用层的缓存系统（Cache）、数据库只读副本（Read Replica）、CQRS 架构中的事件投影视图（Event Projection），还是跨数据中心的多活部署，均可在该原则指导下系统界定权威属主或冲突消除策略。

## 10.3 条件性情境：实现成本降低时的瓶颈变化

设某类工具降低特定任务的单位实现成本  $c_{\text{impl}}$ ，并将其余成本暂时保持不变。这是一个参数情境，不是关于现有生成工具在所有项目中提效的经验结论。成本下降也不等于资源影子价格  $\lambda_{\text{impl}}$  必然趋于零：后者由预算是否紧张以及资源对最优价值的边际作用决定。

若由此增加的候选代码需要相同或更多审查、集成和维护，瓶颈可能转向这些环节；若验证也同时改善，则需要共同重新估计。代码量和提交数不能单独代表产出价值，多维生产力度量可提供补充 [28]。这一情境下，KRCIGE 要求同时记录生成、验证、运行和后续修改成本，以及执行权限与恢复对象。

对于能够直接改变外部状态的自动化系统，审批规则、隔离执行、事务边界和恢复演练都是候选控制措施。其效果依赖观测完整性、规则正确性以及执行过程不绕过检查等前提。确定性拦截器也不能仅因自身确定就保证系统安全，仍需检查系统级交互和约束落实 [26]。这些措施是否值得投入，应由风险、有效性及成本的具体比较决定。

## 11 理论边界与要素区分

为防止将异质性的工程约束混淆为笼统的“工程困难”，下表通过反例对各核心要素的操作化边界予以概念区分：

| 区分       | 反例                                                                                |
|----------|-----------------------------------------------------------------------------------|
| K / R    | 不可观测的隐状态即便在计算资源充裕时依然无法获知；完全可观测的确定性有限状态机在状态转移时仍持续消耗时间、内存与能量。                       |
| K / C    | 决策主体可知悉外部气象变化却无力改变气象；反之，主体无需获知硬件寄存器的历史残留值即可强制将其重置为初始目标状态。                         |
| I / K    | 可以事先知道某项删除的后果，却在任务相关状态及全部恢复证据被擦除后无法精确恢复；也可以不了解新特性的业务表现，但通过隔离和完整恢复路径降低 I。          |
| I / R    | $I$ 给出系统达成任务等价恢复所需的最小客观成本负担， $R$ 界定主体当前可支配的资源预算上界；面对相同的恢复任务，不同预算规模的主体呈现出截然不同的可行性。 |
| I / C    | 控制边界 $C$ 界定了决策主体可执行的恢复动作策略集 $\Pi_E$ ；而第三方接口依赖、外部人员行为与网络传播则直接改变了任务恢复的客观成本 $I$ 。    |
| I / 技术债务 | 维护负担较高的遗留模块可以具备低成本回退路径；维护负担较低的公共 API 也可能因外部依赖而需要高成本迁移。两者可重叠或相关，见注 5.7。            |

前文关于多对一变换的论证，进一步从状态空间塌缩与信息擦除的角度给出了不可逆性产生的确定性机理。

K、R、C、I 的取值既依赖物理与计算条件，也受任务、权限、证据和组织安排影响。G 与 E 分别规定评价方向与允许边界。凡涉及认知、组织或工具效果的解释，都需要具体经验前提；附录的  $\Psi$  标记只指出尚需补充证据的位置，不能把未说明的机制自动视为已被框架解释。

## 12 实证检验设计

当前证据包括形式化模型、条件命题、合成案例与已有实践的定性映射。它们分别支持内部一致性、模型内的策略计算和概念对应，尚不足以支持独立预测准确率、因果效应或真实团队使用收益。以下先报告现有材料的性质，再给出拟开展的评估。

### 12.1 现有材料的解释性对照

*The Pragmatic Programmer* 100 Tips 是框架构造与解释材料，见附录 A。Twelve-Factor App 提供另一组工程实践的对应分析 [11]，见附录 B。当前项目未提供足以核验的预注册记录、带时间戳的冻结预测、多名独立编码结果或盲评过程，因此不将该分析称为前瞻留出验证，也不声称已执行盲审。

现有表格沿用 10 项“直接对应”和 2 项“部分对应”的工作分类。它们描述文本中机制的相似程度，不是预测正确率，也未证明变化方向具有因果支持。框架与映射在修订中相互影响，材料选择和多标签解释具有自由度；未观察到反例也可能来自选择方式。后续独立材料需另行采集，已有对应表不得重新计为未接触过的测试集。

## 12.2 独立评估协议

第一类研究检验方向预测与边界判断：先冻结框架版本、少量具体假设、变量定义、桥接前提  $\Psi_j$ 、数据纳入规则及评分方法，再按项目或组织划分开发集和测试集，必要时采用时间顺序留出。同一项目的相邻发布、同一事故的不同报告或文档的多个段落不能同时跨入训练与测试，避免信息泄漏。

至少两名编码者在看不到结果变量和其他编码者判断的情况下，依据同一手册编码决策前条件；独立评估结果后再讨论分歧，并记录无法编码的案例。可报告 Cohen's  $\kappa$  或 Krippendorff's  $\alpha$  及其不确定性，但一致性只说明测量能否重复，不证明模型正确。事故样本应覆盖成功、失败和边界情形，并说明报告缺失及幸存者偏差。

比较基线至少包括常用工程启发、使用相同可得数据的项目风险模型，以及加入预先规定 KRCIGE 变量的模型。若基础模型已包含恢复成本等信息，应避免把换名后的同一变量当作新增信息。拟合预算、特征选择及调参机会应可比。

对于二元预测，记录每次决策前给出的概率  $\hat{p}_i$  与实际结果  $y_i$ ，使用

$$BS = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2, \quad \Delta BS = BS_{\text{baseline}} - BS_{\text{KRCIGE}}.$$

Brier 分数和对数损失评价概率预测的综合质量 [33]； $1 - BS$  不能单独称作校准度。校准应另行检查预测概率与发生频率的一致性，例如报告可靠性图及区间。边界识别同时报告检出率和误报率；连续结果可以使用预先指定的预测误差。效应区间与重采样应以项目或组织等独立单元为依据，避免把相关发布当作独立样本。

第二类研究直接检验决策辅助效果：在同等资料、时间预算和求解工具下，比较使用现有决策方法与增加 KRCIGE 情境记录的团队。优先考虑随机分配或平衡顺序的受控任务，记录遗漏约束数量、恢复方案可行性、决策后的任务结果及建模工时；需要由独立评估者判断结果，不能仅以“是否使用 KRCIGE 术语”评分。已有方法中包含同样信息时，应期待相同计算结果，并把研究重点放在信息发现与使用成本上。

预测误差下降或  $\Delta R^2$  改善不自动提供因果识别。若只能获得观察性项目数据，需要预先说明混杂、处理选择和目标因果量，并把无法排除的替代解释作为局限。数据、编码手册和分析代码在隐私、权限及共享条件允许范围内公开；受限数据至少公开数据字典、合成样例及可复算分析流程。

## 12.3 可反驳条件

一个具有生命力的科学理论必须具备明确的可反驳性 (Falsifiability)。以下情况分别可能要求修正某项机制、缩小适用范围，或否定框架的增量价值；单个局部假设失效不自动推翻全部概念体系：

1. **理论维度无序膨胀**：对核心工程原则的解释，必须持续引入彼此无关的全新底层基础约束，表明现有约束体系不具备结构完备性；



2. **内部逻辑自相矛盾**: 在相同的基础条件与参数设定下, 框架频繁推导出互相冲突且无法通过成本影子价格、效用目标或系统分层予以消解的对立策略;
3. **方向预测系统性背离**: 框架关于原则适用边界与比较静态学单调变化方向的推导, 在统计检验中长期且系统性地与工业实证结果相背离;
4. **概念压缩优势丧失**: 为消除预测偏差, 中间原则  $P_i$  的数量无序膨胀并逼近经验原则的原始总数, 导致理论丧失科学模型所必需的信息压缩能力;
5. **辅助假设事后泛化**: 辅助经验前提  $\Psi$  或可学习条件  $\Sigma$  被滥用为包罗万象的事后修补工具, 使得任意反例均可通过追加辅助假设被无代价吸收, 导致理论沦为不可证伪的循环论证。

上述显式列出的反驳准则, 旨在从方法学上严格约束框架的模型复杂度, 限制事后解释与辅助假设膨胀。

## 12.4 可量化研究方向

应先选择少量可操作问题, 而不是一次检验全部原则。例如, 在已明确回滚成功标准和同类故障条件的发布中, 比较恢复证据与演练投入是否改变可行恢复成本; 在候选行动集合固定时, 比较反馈模型是否准确预测“继续观察”与“停止”的边界。演练得到的是已执行策略的成本和成功率样本, 不能直接当作未知全局最优恢复成本。

每项研究需预先规定预测方向、效果量区间和失败条件。可检验的局部预测包括: 有效防护按固定比例降低恢复成本时, 预期防护收益随原恢复成本上升; 恢复预算越过硬门槛时, 某些候选行动进入或退出可行集。相反,  $k_Q$  增大并不自动使任意探针更有价值,  $\gamma_Q$  增大也不保证任意冗余方案有效。控制成本、相关故障、测量误差和决策替代路径应在模型中明确。

# 13 应用范围与使用流程

## 13.1 适用边界与跨层接入

本文主要讨论接口、数据、模块边界、测试、发布与恢复等技术决策。框架可以提示这些决策涉及哪些组织和社会条件, 但尚未提供关于团队行为、激励分配或目标协商的完整解释模型。存在利益冲突、战略互动或持续变化的任务时, 应结合相应领域方法重新界定情境, 不能仅添加一个  $\Psi$  或  $E$  标签就认定问题已被解决。

## 13.2 工程决策实施清单

在面对具体的重大技术选型、架构演进或研发流程重构时, 工程师与架构师可依托以下结构化评估清单, 逐项厘清系统的约束边界:

1. **情境定界 (Context)**: 明确当前求解的核心任务  $Q$ 、决策主体  $s$  以及有限评估时间窗口  $\tau$ ; 严谨界定候选行动空间  $\mathcal{A}_s$ 、状态转移规则以及预期的目标状态集合  $\mathcal{S}_G$ 。
2. **认知边界评估 (K)**: 记录影响候选策略排序的未知参数、现有证据与估计区间; 区分完整信息的价值上限  $k_Q$  和实际可用探针的净价值。
3. **资源预算核算 (R)**: 研发人力、计算资源、存储容量与前置时间预算的瓶颈位于何处? 各维度的资源影子价格  $\lambda_j$  如何赋值?
4. **控制权限审视 (C)**: 决策主体的有效控制范围与执行权限为何? 外部不可控扰动集合  $\mathcal{D}$  及相应概率模型如何界定? 控制缺口  $\gamma_Q$  反映怎样的风险下界? 若任务规定风险上限, 实际候选策略是否满足  $r_Q(\pi | h_t, b_t) \leq g_{\max}$ ?

5. **不可逆性推演 (I)**: 明确恢复代码、数据、新写入及外部后果中的哪些对象, 记录容差、成功概率、证据、允许策略与预算; 区分实测恢复成本、可达上界和未知最优值。
6. **优化目标对齐 (G)**: 记录受益者、风险承担者及后续维护者, 明确目标权重、折现与终止残值的确定方式; 目标未达成一致时保留多目标比较。
7. **边界落实 (E)**: 记录允许行动的约束来源、检查证据、执行责任及变更机制, 并检查局部措施能否覆盖系统级交互。
8. **学习反馈校准 ( $\Sigma$ )**: 先说明信号模型、适用窗口与迁移证据, 再比较观测是否改变可行选择以及净价值是否为正。
9. **预验与反思机制**: 为采纳的工程策略显式预设量化评估指标、变化方向与反转边界, 并确立定期的事后复盘与架构回顾机制。

记录完成后, 团队可以比较少量候选方案, 并保留尚未解决的分歧和估计误差。对于低影响、易恢复的任务, 可使用简化记录; 分析本身的成本也应与决策影响相称。

## 14 结论

KRCIGE 将知识、资源、控制和恢复负担组织为软件工程决策的四个检查方向, 并以目标、允许边界与反馈结构限定比较范围。本文的主要工作是给出共同的情境语言, 明确任务相关恢复成本的定义与边界, 连接十三条条件性工程机制, 并提供可复算的发布案例。信息价值、实物期权、恢复计算与系统安全构成其已有研究基础。

本轮案例表明, 同一机制在损失、反馈成本或恢复预算改变后, 可以导向观察、扩大或停止; 尤其是观察的增量价值并不随恢复成本单调增加。文本映射只支持解释性对应, 不能承担前瞻性预测或因果验证。技术债务与 I 具有不同的提问焦点, 但可相互重叠; 单主体最优也不能替代跨团队、跨时间的目标与风险协调。

下一阶段应以少量预先登记的决策问题检验框架的实际增量: 它是否帮助团队发现遗漏的恢复要求和利益相关者, 能否更准确地识别策略边界, 以及这些改进是否覆盖额外建模成本。独立数据、可比基线和明确失败条件, 比继续扩展原则数量更能决定框架的研究价值。

### A 对《The Pragmatic Programmer》100 条 Tips 的映射

*The Pragmatic Programmer* (《程序员修炼之道》) 20 周年版系统整理并公开发布了 100 条经典工程实践经验格言 (Tips) [9, 10]。本节将该格言集作为构造语料, 展示 KRCIGE 对工程实践的解释性映射。表中所列的“推导路径”代表基于物理约束与派生原则的机理映射; 凡涉及个体认知心理学、团队组织行为学或特定工程文化的条目, 均显式标记辅助经验前提  $\Psi$ , 以严谨划定理论核心与经验假设的边界。

为便于阅读, 使用以下缩写:

$P_1$  = 反馈价值,     $P_2$  = 选择权,     $P_3$  = 复杂度预算,  
 $P_4$  = 局部化,     $P_5$  = 显式假设,     $P_6$  = 信息保留,  
 $P_7$  = 权威来源,     $P_8$  = 自动化,     $P_9$  = 状态空间,  
 $P_{10}$  = 可恢复性,     $P_{11}$  = 安全约束,     $P_{12}$  = 知识外部化,  
 $P_{13}$  = 模型更新.

表 1: 100 条 Tips 与 KRCIGE 的初步推导路径

| #  | Tip (中文概括)                                | 路径                                                                               |
|----|-------------------------------------------|----------------------------------------------------------------------------------|
| 1  | 认真打磨自己的专业能力                               | $K + R + G + \Psi \rightsquigarrow P_{12} \rightsquigarrow$ 持续提高能力与知识质量。         |
| 2  | 思考你的工作                                    | $K + \Sigma + G \rightsquigarrow P_1 + P_{13} \rightsquigarrow$ 持续反思并更新工作模型。     |
| 3  | 你拥有主动权                                    | $C + G \rightsquigarrow$ 识别可控行动子集 $\rightsquigarrow P_{13}$ 。                    |
| 4  | 提供可选方案, 别拿蹩脚借口搪塞                          | $C + K + G \rightsquigarrow P_2 \rightsquigarrow$ 在可控范围提供选择与代价。                  |
| 5  | 不要容忍已经存在的缺陷、糟糕设计和错误决定                     | $C + I + R + G \rightsquigarrow P_{10} + P_3 \rightsquigarrow$ 趁上下文与修复成本仍低时处理退化。 |
| 6  | 主动推动有益的改变发生                               | $C + G + \Psi \rightsquigarrow$ 从可控局部形成反馈与扩散。                                    |
| 7  | 持续关注系统的整体目标和影响                            | $K + R + G \rightsquigarrow P_4 \rightsquigarrow$ 局部决策同时检查系统级目标。                 |
| 8  | 把质量作为明确的需求来讨论                             | $K + R + G \rightsquigarrow P_3 + P_{13} \rightsquigarrow$ 将质量、成本与价值显式建模。        |
| 9  | 持续投资你的知识和技能                               | $K + C + R + G \rightsquigarrow P_{12} + P_{13}$ 。                               |
| 10 | 批判性地分析你读到和听到的内容                           | $K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。                                    |
| 11 | 像编写代码一样清晰、简洁、可维护地使用自然语言                   | $K + I \rightsquigarrow P_5 + P_{12} \rightsquigarrow$ 语言同样承载可丢失的工程语义。           |
| 12 | 你说什么重要, 你怎么说同样重要                          | $K + I \rightsquigarrow P_6 + P_{12}$ 。                                          |
| 13 | 把文档直接构建到系统和工作流中                           | $K + I + R + C \rightsquigarrow P_{12} + P_8$ 。                                  |
| 14 | 好设计比坏设计更容易修改                              | $K + I + R + C + G \rightsquigarrow P_2 + P_4$ 。                                 |
| 15 | DRY——避免重复表达同一份知识                          | $R + C + I + G \rightsquigarrow P_7$ 。                                           |
| 16 | 让复用变得容易                                   | $R + G \rightsquigarrow P_3 + P_7 + P_4$ 。                                       |
| 17 | 消除无关事物之间的影响                               | $K + R + C + G \rightsquigarrow P_4$ 。                                           |
| 18 | 尽量保留调整和撤销决策的空间                            | $K + I + G \rightsquigarrow P_2$ 。                                               |
| 19 | 根据实际问题和证据选择技术, 避免盲目追逐潮流                   | $K + R + \Sigma + G \rightsquigarrow P_1 + P_3 + P_{13}$ 。                       |
| 20 | 尽早构建可运行的局部方案, 用实际结果验证方向                   | $K + C + \Sigma + R + G \rightsquigarrow P_1 + P_2$ 。                            |
| 21 | 用原型验证理解、需求和技术方案                           | $K + \Sigma + R + I + G \rightsquigarrow P_1 + P_2$ 。                            |
| 22 | 让程序贴近真实业务和问题背景                            | $K + I + R \rightsquigarrow P_5 + P_{12} + P_3$ 。                                |
| 23 | 用估算提前暴露不确定性和风险                            | $K + R + G \rightsquigarrow P_{13} + P_3$ 。                                      |
| 24 | 根据代码验证结果持续调整进度计划                          | $K + C + \Sigma + G \rightsquigarrow P_{13}$ 。                                   |
| 25 | 用纯文本保存知识                                  | $I + K + G \rightsquigarrow P_6 + P_{12} + P_2$ , 具体介质选择依赖 $\Psi$ 。              |
| 26 | 用命令 Shell 组合和自动化工作流程                      | $R + G + \Psi \rightsquigarrow P_8 + P_3$ 。                                      |
| 27 | 熟练掌握编辑器, 让编辑操作高效而自然                       | $R + G + \Psi \rightsquigarrow P_3$ 。                                            |
| 28 | 始终使用版本控制                                  | $C + I + R + G \rightsquigarrow P_6 + P_{10} + P_8$ 。                            |
| 29 | 修复问题, 别追究责任                               | $R + K + G + \Psi \rightsquigarrow P_1 + P_{12}$ 。                               |
| 30 | 保持冷静                                      | $R + G + \Psi \rightsquigarrow$ 保护有限认知资源。                                        |
| 31 | 先写一个会失败的测试, 再修复代码                         | $K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。                                    |
| 32 | 仔细读懂错误消息                                  | $K + I + G \rightsquigarrow P_6 + P_1$ 。                                         |
| 33 | 先检查查询、数据和假设, 别急着认为 <code>select</code> 出错 | $K + \Sigma + \Psi \rightsquigarrow P_1$ , 结合成熟组件的经验先验。                          |

| #  | Tip (中文概括)                      | 路径                                                            |
|----|---------------------------------|---------------------------------------------------------------|
| 34 | 别想当然, 用证据验证                     | $K + \Sigma + G \rightsquigarrow P_1。$                        |
| 35 | 学一门文本处理语言                       | $R + G + \Psi \rightsquigarrow P_8 + P_3。$                    |
| 36 | 承认软件不可能完美, 持续改进它                | $K + R + C + G \rightsquigarrow P_3。$                         |
| 37 | 用明确的前置条件、后置条件和不变量设计代码           | $K + C + I \rightsquigarrow P_5。$                             |
| 38 | 尽早发现严重错误并停止执行, 避免继续产生错误结果       | $C + I + G \rightsquigarrow P_5 + P_6 + P_{10}。$              |
| 39 | 用断言检查不变量, 尽早捕获不应发生的状态           | $K + C + \Sigma \rightsquigarrow P_5 + P_1。$                  |
| 40 | 完成已经开始的工作, 或明确结束它的方式            | $R + I + G + \Psi \rightsquigarrow P_3 + P_{12}。$             |
| 41 | 把变化和影响限制在局部范围内                  | $K + R + C + G \rightsquigarrow P_4。$                         |
| 42 | 每次只做小幅、可验证的改动                   | $K + C + I + \Sigma + G \rightsquigarrow P_1 + P_2 + P_{10}。$ |
| 43 | 不要假设未来需求和技术会怎样, 依据当前证据做可调整的决策   | $K + I + R + G \rightsquigarrow P_2 + P_3。$                   |
| 44 | 减少模块之间的依赖, 让代码更容易修改             | $K + R + C + G \rightsquigarrow P_4。$                         |
| 45 | 让对象直接执行操作, 通过行为接口协作, 减少对内部状态的询问 | $K + R + C \rightsquigarrow P_4 + P_5。$                       |
| 46 | 避免让方法调用形成过长的链条                  | $K + R + C \rightsquigarrow P_4。$                             |
| 47 | 避免全局数据                          | $K + C + R \rightsquigarrow P_9 + P_4。$                       |
| 48 | 通过 API 管理需要全局共享的重要资源            | $K + C + I \rightsquigarrow P_5 + P_4 + P_9。$                 |
| 49 | 编程要写代码, 但程序最终处理的是数据             | $K + I + G \rightsquigarrow P_5 + P_6。$                       |
| 50 | 减少状态的集中持有, 让状态沿数据流传递            | $K + C + I \rightsquigarrow P_5 + P_9。$                       |
| 51 | 警惕继承带来的耦合和维护成本                  | $R + K + C + G \rightsquigarrow P_3 + P_4。$                   |
| 52 | 优先用接口表达不同实现之间的多态关系              | $K + I + G \rightsquigarrow P_2 + P_4。$                       |
| 53 | 优先通过服务委托和组合复用功能, 减少对继承的依赖       | $K + I + R + G \rightsquigarrow P_2 + P_4 + P_3。$             |
| 54 | 用 mixin 共享可复用功能                 | $R + G + \Psi \rightsquigarrow P_7 + P_4。$                    |
| 55 | 让应用通过外部配置适配不同环境                 | $K + C + I + G \rightsquigarrow P_2 + P_5 + P_9。$             |
| 56 | 分析 workflow, 找出可以并发执行的部分        | $R + K + \Sigma + G \rightsquigarrow P_1 + P_9。$              |
| 57 | 尽量避免共享可变状态, 因为它容易导致不一致和并发错误     | $K + R + C \rightsquigarrow P_9。$                             |
| 58 | 随机失败往往是并发问题                     | $C + K + \Psi \rightsquigarrow P_1 + P_9。$                    |
| 59 | 使用 Actor 模型, 在无共享状态条件下实现并发      | $K + R + C \rightsquigarrow P_9 + P_4。$                       |
| 60 | 用共享的信息空间协调 workflow             | $K + C + I + \Psi \rightsquigarrow P_5 + P_9 + P_4。$          |
| 61 | 识别并管理面对风险和批评时的本能防御反应            | $K + R + \Psi \rightsquigarrow P_1$ , 将直觉作为待验证信号。             |

| #  | Tip (中文概括)                | 路径                                                             |
|----|---------------------------|----------------------------------------------------------------|
| 62 | 不要依赖碰巧成立的行为、顺序或环境来编程      | $K + \Sigma + G \rightsquigarrow P_1 + P_5。$                   |
| 63 | 估算算法随输入规模增长的资源消耗          | $R + G \rightsquigarrow P_3。$                                  |
| 64 | 用实际测试检验对算法性能的估算           | $K + \Sigma + G \rightsquigarrow P_1 + P_{13}。$                |
| 65 | 尽早重构, 持续重构                | $K + R + C + I + G \rightsquigarrow P_2 + P_3 + P_4 + P_{12}。$ |
| 66 | 测试用于验证行为、保护回归, 并发现 Bug    | $K + I + \Sigma \rightsquigarrow P_5 + P_{12}。$                |
| 67 | 先用测试验证代码的使用方式和接口          | $K + R + G \rightsquigarrow P_1 + P_4。$                        |
| 68 | 先构建贯穿完整流程的可运行方案, 再逐步扩展    | $K + C + \Sigma + G \rightsquigarrow P_1。$                     |
| 69 | 为可测试性而设计                  | $K + C + \Sigma + G \rightsquigarrow P_1 + P_4 + P_5。$         |
| 70 | 测试你的软件, 避免把缺陷暴露给用户        | $K + C + \Sigma + G \rightsquigarrow P_1。$                     |
| 71 | 用基于属性的测试验证对系统行为的假设        | $K + \Sigma + G \rightsquigarrow P_1 + P_5。$                   |
| 72 | 保持简单, 减少可被攻击的入口           | $R + C + I + E \rightsquigarrow P_3 + P_{11}。$                 |
| 73 | 快速应用安全补丁                  | $C + I + E + G \rightsquigarrow P_{10} + P_{11}。$              |
| 74 | 选择清晰准确的名称, 需要时及时重命名       | $K + I + G \rightsquigarrow P_5 + P_{12}。$                     |
| 75 | 承认需求一开始通常并不明确             | K 的需求领域实例。                                                     |
| 76 | 帮助用户把模糊需求说清楚              | $K + \Sigma + G + \Psi \rightsquigarrow P_1 + P_{13}。$         |
| 77 | 通过反复收集反馈和调整实现来逐步明确需求      | $K + C + \Sigma + G \rightsquigarrow P_1 + P_{13}。$            |
| 78 | 和用户一起工作, 站在用户角度思考         | $K + \Sigma + G \rightsquigarrow P_1 + P_{12}。$                |
| 79 | 把变化频繁的策略作为可配置的元数据管理       | $K + C + I + G \rightsquigarrow P_2 + P_5 + P_9。$              |
| 80 | 建立并使用统一的项目术语表             | $K + I + R \rightsquigarrow P_5 + P_{12}。$                     |
| 81 | 先识别问题和约束边界, 再寻找突破方式       | $K + \Sigma + G \rightsquigarrow P_1 + P_{13}。$                |
| 82 | 通过结对编程、代码评审或团队协作来编写代码     | $K + R + I + \Psi \rightsquigarrow P_{12}。$                    |
| 83 | 把敏捷落实为持续反馈、快速交付和持续改进的做事方式 | $K + C + \Sigma + I + G \rightsquigarrow P_1 + P_2 + P_{13}。$  |
| 84 | 保持团队规模小而稳定                | $R + I + \Psi \rightsquigarrow P_3 + P_{12}。$                  |
| 85 | 把重要的改进和学习安排进日程, 才能落实      | $R + G + \Psi \rightsquigarrow$ 显式资源配置。                        |
| 86 | 组建能够独立完成端到端工作的团队          | $K + R + C + G + \Psi \rightsquigarrow P_4 + P_{12}。$          |
| 87 | 根据实际效果做事, 别追逐时髦           | $K + R + \Sigma + G \rightsquigarrow P_1 + P_3 + P_{13}。$      |
| 88 | 在用户真正需要时交付价值              | $R + C + G \rightsquigarrow P_{13}$ , 价值时点由 G 给出。              |
| 89 | 让版本控制触发构建、测试和发布           | $C + I + R + G \rightsquigarrow P_6 + P_8 + P_{10}。$           |

| #   | Tip (中文概括)                     | 路径                                                                   |
|-----|--------------------------------|----------------------------------------------------------------------|
| 90  | 尽早测试、频繁测试，并自动执行测试              | $K + C + R + \Sigma + G \rightsquigarrow P_1 + P_5 + P_8$ 。          |
| 91  | 所有测试跑完，编码才算完成                  | $K + \Sigma + G \rightsquigarrow P_1 + P_5$ ，完成定义来自 $G$ 。            |
| 92  | 用故意制造错误的测试程序检验测试               | $K + \Sigma + G \rightsquigarrow P_1 + P_5$ 。                        |
| 93  | 以状态覆盖率检验测试，别只看代码覆盖率            | $K + I + \Sigma \rightsquigarrow P_5 + P_9$ 。                        |
| 94  | 每个已发现的 Bug 都要用自动化测试固定下来，避免再次出现 | $K + I + R + G \rightsquigarrow P_6 + P_{12} + P_5$ 。                |
| 95  | 把手工流程自动化                       | $R + C + G \rightsquigarrow P_8$ 。                                   |
| 96  | 让用户获得满意和惊喜，交付可用价值              | $K + \Sigma + G + \Psi \rightsquigarrow P_1 + P_{13}$ 。              |
| 97  | 为自己的成果署名并承担责任                  | $G + E + \Psi \rightsquigarrow$ 责任与可追溯性；KRCI 可解释其工程收益。               |
| 98  | 优先确保软件不造成伤害                    | $E \rightsquigarrow P_{11}$ ； $C + I$ 提高高损害行动的谨慎级别。                  |
| 99  | 不要为恶意使用者提供便利                   | $E + C + I \rightsquigarrow P_{11}$ 。                                |
| 100 | 分享成果，庆祝进展，持续改进和建设              | $K + I + R + G + \Psi \rightsquigarrow P_{12} + P_{13}$ ；庆祝属于组织文化前提。 |

上述映射关系展现了中间原则 P1–P13 与基础约束在经典工程语料中的高度复用性；该表格主要用于检验理论框架的概念压缩与解释覆盖能力，不单独承担独立的实证反驳功能。

## B Twelve-Factor App 解释性对照

本附录保留对 Twelve-Factor App 的定性机制映射。它是在框架形成与修订过程中整理的解释性对照，尚未经过独立盲态编码；“直接对应”只表示材料直接提及相关机制，不意味着比较静态或因果效果得到验证。

表 2: Twelve-Factor App 的定性机制对应

| # | 对照因素             | 解释路径与待检验方向                                 | 官方说明对应与工作分类                                         |
|---|------------------|--------------------------------------------|-----------------------------------------------------|
| 1 | Codebase         | $P7 + P6$ ；部署分叉增多时，权威来源与版本历史价值上升。          | 官方要求一个代码库对应一个应用、由版本控制追踪并支持多个部署，覆盖权威来源和历史保留。分类：直接对应。 |
| 2 | Dependencies     | $P5 + P7 + P10$ ；执行环境差异越大，显式声明和隔离的价值越高。    | 官方要求完整声明、隔离依赖，并以未来机器可能缺包或版本不兼容解释其价值。分类：直接对应。        |
| 3 | Config           | $P2 + P5 + P9$ ；部署间变化越频繁，配置与代码分离的价值越高。     | 官方把配置定义为部署间变化项，要求独立、正交管理，并讨论组合爆炸。分类：直接对应。           |
| 4 | Backing services | $P4 + P5 + P10$ ；外部服务替换和故障概率上升时，附着式边界价值上升。 | 官方要求以资源句柄连接服务，并给出数据库故障后从备份恢复、重新挂接且无需改代码的例子。分类：直接对应。 |

| #  | 对照因素                | 解释路径与待检验方向                                    | 官方说明对应与工作分类                                                          |
|----|---------------------|-----------------------------------------------|----------------------------------------------------------------------|
| 5  | Build, release, run | $P4 + P6 + P10$ ；发布恢复要求越高，阶段分离和不可变发布账本越有价值。   | 官方严格区分三个阶段，要求发布具有唯一 ID、保持追加式不可变，并明确支持快速回滚。分类：直接对应。                   |
| 6  | Processes           | $P9 + P4 + P10$ ；重启和调度越不可控，无共享、无状态进程的价值越高。    | 官方要求进程无状态且无共享，并以重启、迁移和不同进程处理后续请求说明本地状态不可靠。分类：直接对应。                   |
| 7  | Port binding        | $P4 + P5$ ；运行容器差异增大时，自包含服务和显式端口契约价值上升。        | 官方支持自包含服务和端口契约，机制吻合；详细说明没有直接比较容器差异变化时的策略强度。分类：部分对应。                  |
| 8  | Concurrency         | $P9 + P3 + P13$ ；负载与工作类型增加时，按进程类型分解和横向扩展价值上升。 | 官方以进程类型表达工作差异，以无共享和水平分解释可靠扩展，并讨论纵向扩展限制。分类：直接对应。                      |
| 9  | Disposability       | $P10 + P4$ ；终止和迁移越频繁，快速启动、优雅退出与幂等价值上升。        | 官方直接以弹性扩缩、部署、硬件突然失效和任务重入说明快速启动、优雅退出与幂等。分类：直接对应。                      |
| 10 | Dev/prod parity     | $P1 + P5 + P13$ ；环境差异越大，生产特有错误和反馈延迟越高。        | 官方列出时间、人员和工具三类差距，并说明后端服务差异会使测试通过的代码在生产失败。分类：直接对应。                    |
| 11 | Logs                | $P1 + P6 + P12$ ；运行时不确定性越高，事件证据的诊断与历史价值越高。    | 官方把日志定义为事件流，并明确用于历史检索、趋势分析和主动告警。分类：直接对应。                             |
| 12 | Admin processes     | $P4 + P8 + P10$ ；一次性高风险任务应与常驻流程分离并沿用可恢复发布路径。  | 官方支持一次性进程及相同代码、配置和依赖，机制部分吻合；其主要论证更集中于环境一致性和同步，未直接给出不可逆性强度关系。分类：部分对应。 |

表中保留的工作分类为 10 项直接对应、2 项部分对应，尚未有独立编码者复核。该数量仅描述当前映射，不能换算为预测命中率。

## C P1–P13 的可检验假设记录

本附录汇总正文第 7 节的候选假设记录。各项机制尚需在具体任务中给出效果量、测量误差和经验前提，不能仅凭本表认定已实现标准化测量。

表 3: P1–P13 的可检验假设记录

| 原则 | 机制         | 主要指标            | 方向与适用边界                             |
|----|------------|-----------------|-------------------------------------|
| P1 | 信息改变可行选择   | 净信息价值、决策变更、预测误差 | 信号可迁移且会改变行动时比较净值；昂贵、失真或行动不变的观测可能无益。 |
| P2 | 后续选择抵消锁定损失 | 可选方案、保留和等待成本    | 机会仍存在且额外选择收益覆盖总成本；截止期与维护负担可反转方向。    |
| P3 | 复杂度占用多类资源  | 实现、理解、运行和变更成本   | 评价全周期净价值；减少一处复杂度可能把成本转移至另一处。        |

| 原则  | 机制         | 主要指标             | 方向与适用边界                            |
|-----|------------|------------------|------------------------------------|
| P4  | 边界限制传播与协调  | 故障及变化传播范围、边界成本   | 隔离有效且边界成本可控；共同原因故障不一定被模块边界隔开。      |
| P5  | 假设显式化支持核验  | 假设相关缺陷、检查与维护成本   | 契约反映真实要求且持续维护；错误契约可能强化共同误解。        |
| P6  | 证据支持诊断与恢复  | 检索成本、证据充分性、恢复成功率 | 记录可用且收益覆盖存储、检索与边界成本；数据多不等于证据充分。    |
| P7  | 权威或协议协调事实  | 冲突、恢复与协调代价       | 评价更新语义和失效模式；单一权威也可能产生瓶颈或单点故障。      |
| P8  | 固定投入降低重复消耗 | 自动化总成本、失败率、执行次数  | 工作可重复且规则稳定；新增维护与相关故障成本进入比较。        |
| P9  | 约束可达状态和交错  | 状态空间、协调与运行成本     | 在保持任务表达能力时减少无用状态；无状态进程可能将状态转移至数据库。 |
| P10 | 预防和恢复降低损失  | 条件恢复成本、演练成功率     | 使用已验证的效果与共同恢复目标；无法处理新写入的回滚不等于任务恢复。 |
| P11 | 权限与隔离限制损害  | 权限范围、失效损失、执行证据   | 限制实际可执行路径且覆盖相关威胁；安全目标还取决于系统交互。     |
| P12 | 外部载体支持知识继承 | 交接和检索工时、更新率      | 内容可信可用且维护成本可控；过时文档可能产生负效应。         |
| P13 | 证据支持模型修正   | 预测质量、更新延迟与成本     | 比较信号可信度及重构代价；需求与目标的变更需保留版本和决策理由。   |



## D 符号表

| 符号                        | 含义                      |
|---------------------------|-------------------------|
| $Q, s, \tau$              | 工程任务、主体与有限评估时间窗         |
| $S_Q$                     | 工程情境及其状态、观测、行动、预算与伦理边界  |
| K                         | Knowledge bounded: 认识有限 |
| R                         | Resources bounded: 资源有限 |
| C                         | Control bounded: 控制有限   |
| I                         | Irreversibility: 工程不可逆性 |
| $k_Q$                     | 决策相关认识缺口, 即完整信息的期望决策价值  |
| $\lambda_j$               | 资源 $j$ 的预算边际价值或影子价格     |
| $r_Q(\pi   h_t, b_t)$     | 具体可行策略在给定扰动集合下的最坏期望风险   |
| $\gamma_Q$                | 可行策略最坏期望风险的下确界, 即控制缺口   |
| $g_{\max}$                | 任务规定的最坏期望风险上限           |
| $\Pi_Q^g, V_Q^g$          | 满足策略级风险上限的可行策略集及其最优值函数  |
| $I_{\varepsilon, \delta}$ | 达到任务等价恢复要求的最低预期成本       |
| NVOI                      | 观测扣除获取成本后的净决策价值         |
| G                         | Goal: 工程目标              |
| E                         | Ethics: 伦理边界            |
| $\Sigma$                  | 可学习结构条件                 |
| $\Psi$                    | 心理学、组织行为与具体工具经验等辅助实证前提  |

## 参考文献

- [1] Herbert A. Simon. “Rational Decision-Making in Business Organizations.” Nobel Memorial Lecture, 1978. <https://www.nobelprize.org/prizes/economic-sciences/1978/simon/lecture/>
- [2] Ronald A. Howard. “Information Value Theory.” *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966. DOI: <https://doi.org/10.1109/TSSC.1966.300074>
- [3] W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, 1956. <https://ashby.info/Ashby-Introduction-to-Cybernetics.pdf>
- [4] Rolf Landauer. “Irreversibility and Heat Generation in the Computing Process.” *IBM Journal of Research and Development*, 5(3):183–191, 1961. DOI: <https://doi.org/10.1147/rd.53.0183>
- [5] David L. Parnas. “On the Criteria To Be Used in Decomposing Systems into Modules.” *Communications of the ACM*, 15(12):1053–1058, 1972. DOI: <https://doi.org/10.1145/361598.361623>

- [6] Harold Abelson, Gerald Jay Sussman, with Julie Sussman. *Structure and Interpretation of Computer Programs*, 2nd ed. MIT Press, 1996. <https://mitpress.mit.edu/9780262510875/structure-and-interpretation-of-computer-programs/>
- [7] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM*, 32(2):374–382, 1985. <https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf>
- [8] David H. Wolpert and William G. Macready. “No Free Lunch Theorems for Optimization.” *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. DOI: <https://doi.org/10.1109/4235.585893>
- [9] David Thomas and Andrew Hunt. *The Pragmatic Programmer: Your Journey to Mastery, 20th Anniversary Edition*. Addison-Wesley, 2019. ISBN 9780135957059. <https://pragprog.com/titles/tpp20/the-pragmatic-programmer-20th-anniversary-edition/>
- [10] The Pragmatic Bookshelf. “Pragmatic Programmer Tips: 100 Tips from the 20th Anniversary Edition.” <https://pragprog.com/tips/>
- [11] Adam Wiggins. *The Twelve-Factor App*. Last updated 2017. <https://12factor.net/>
- [12] Martin Thomson and David Schinazi. “Maintaining Robust Protocols.” RFC 9413, Internet Architecture Board, June 2023. DOI: <https://doi.org/10.17487/RFC9413>
- [13] Avinash K. Dixit and Robert S. Pindyck. *Investment under Uncertainty*. Princeton University Press, 1994.
- [14] Sigurd Skogestad and Ian Postlethwaite. *Multivariable Feedback Control: Analysis and Design*, 2nd ed. Wiley, 2005.
- [15] Erik Hollnagel, David D. Woods, and Nancy Leveson, editors. *Resilience Engineering: Concepts and Precepts*. Ashgate, 2006.
- [16] Ward Cunningham. “The WyCash Portfolio Management System.” In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (OOP-SLA ’92)*, pages 29–30, 1992. DOI: <https://doi.org/10.1145/157709.157715>
- [17] Peter Naur and Brian Randell, editors. *Software Engineering: Report on a conference sponsored by the NATO Science Committee*. Scientific Affairs Division, NATO, Garmisch, Germany, 1969.
- [18] IEEE Computer Society. *IEEE Standard Glossary of Software Engineering Terminology (IEEE Std 610.12-1990)*. IEEE, 1990. DOI: <https://doi.org/10.1109/IEEESTD.1990.101064>
- [19] Barry W. Boehm. *Software Engineering Economics*. Prentice-Hall, 1981.
- [20] Titus Winters, Tom Manshreck, and Hyrum Wright. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O’Reilly Media, 2020.

- [21] Kevin J. Sullivan, Prasad Chalasani, Somesh Jha, and Vibha Sazawal. “Software Design as an Investment Activity: A Real Options Perspective.” In *Real Options and Business Strategy: Applications to Decision Making*, pages 215–262. Risk Books, 1999. Author manuscript: <https://www.researchgate.net/publication/2255701>
- [22] Jayatirtha Asundi, Rick Kazman, and Mark H. Klein. “Using Economic Considerations to Choose Among Architecture Design Alternatives.” Technical Report CMU/SEI-2001-TR-035, Software Engineering Institute, 2001. DOI: <https://doi.org/10.1184/R1/6585749.v1>
- [23] David A. Patterson et al. “Recovery Oriented Computing (ROC): Motivation, Definition, Techniques, and Case Studies.” Technical Report UCB/CSD-02-1175, University of California, Berkeley, 2002. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2002/5574.html>
- [24] Aaron B. Brown and David A. Patterson. “Undo for Operators: Building an Undoable E-mail Store.” In *Proceedings of the 2003 USENIX Annual Technical Conference*, pages 1–14, 2003. <https://www.usenix.org/legacy/events/usenix2003/tech/brown/brown.pdf>
- [25] Paris Avgeriou, Philippe Kruchten, Ipek Ozkaya, and Carolyn Seaman. “Managing Technical Debt in Software Engineering (Dagstuhl Seminar 16162).” *Dagstuhl Reports*, 6(4):110–138, 2016. DOI: <https://doi.org/10.4230/DagRep.6.4.110>
- [26] Nancy G. Leveson. “A New Accident Model for Engineering Safer Systems.” *Safety Science*, 42(4):237–270, 2004. DOI: [https://doi.org/10.1016/S0925-7535\(03\)00047-X](https://doi.org/10.1016/S0925-7535(03)00047-X)
- [27] Paul Ralph. “The Two Paradigms of Software Design.” arXiv preprint arXiv:1303.5938, 2013. <https://arxiv.org/abs/1303.5938>
- [28] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. “The SPACE of Developer Productivity: There’s More to It than You Think.” *ACM Queue*, 19(1):20–48, 2021. DOI: <https://doi.org/10.1145/3454122.3454124>
- [29] Shirley Gregor. “The Nature of Theory in Information Systems.” *MIS Quarterly*, 30(3):611–642, 2006. DOI: <https://doi.org/10.2307/25148742>
- [30] Dag I. K. Sjøberg, Tore Dybå, Bente C. D. Anda, and Jo E. Hannay. “Building Theories in Software Engineering.” In *Guide to Advanced Empirical Software Engineering*, pages 312–336. Springer, 2008. DOI: [https://doi.org/10.1007/978-1-84800-044-5\\_12](https://doi.org/10.1007/978-1-84800-044-5_12)
- [31] Stuart J. Russell and Eric Wefald. “Principles of Metareasoning.” *Artificial Intelligence*, 49(1–3):361–395, 1991. DOI: [https://doi.org/10.1016/0004-3702\(91\)90015-C](https://doi.org/10.1016/0004-3702(91)90015-C)
- [32] Peter D. Grünwald. “A Tutorial Introduction to the Minimum Description Length Principle.” arXiv preprint arXiv:math/0406077, 2004. <https://arxiv.org/abs/math/0406077>
- [33] Tilmann Gneiting and Adrian E. Raftery. “Strictly Proper Scoring Rules, Prediction, and Estimation.” *Journal of the American Statistical Association*, 102(477):359–378, 2007. DOI: <https://doi.org/10.1198/016214506000001437>